P1:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: We're currently using Claude Code. And we're using the paid version of Claude Code — the Team plan.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: Basically, when we do work, normally the planning phase comes first. In the planning phase, first we determine things according to the business logic. Based on that, a PRD is written. After the PRD, we break down the tasks. And once the tasks are broken down, we normally give them to the agents part by part — so modular and proper documentation both are present in our case.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: So, to give an example — we recently worked on something where we needed authentication for one of our services. Since it's a SaaS-type service, the main intention was to implement IAM. But after our implementation, it turned out that even though we told it to implement IAM with role-based authentication, the AI split things into two separate services: one for IAM, and a separate one for authentication, and similar things. Now we've had to backtrack a lot to get things back on track, and redo the proper planning from scratch again because they messed it up.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: The situation was like this — earlier on, when we used to use it, it would make more assumptions. But after the newer, smarter models were introduced, now if something is unclear, it tries to ask a question, and tries to push to get clarification from the business side, or there's more discussion about edge cases. Currently, it doesn't make as many assumptions now, but still — a bit less than before.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: If we talk about understanding, normally we hand over both our planning and the PRD to it. In the PRD, we try to include the business logic and everything — so the task and scope of the project is much more clear. So proper documentation , that’s the way to go actually.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yes, we did do it several times. One of our projects were very rushed. At that time, our implementation was very rushed. We had the project done through AI with just a rough

description, and even then it made a lot of assumptions, and we needed to deliver the thing very quickly. So at that time, it also put in a lot of random functions, features, and validation checks that were actually not needed at all. Later, when we went back to the service for optimization and looked back at it, we found a lot of unnecessary checks and a lot of repetitive API calls, because of which it wasn't actually cost-effective. We had to go back and change that later. Basically, it was more our fault, because we didn't give it proper documentation . It was a rushed project , so we left the whole project up to AI.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: What happens in our case is, normally if I don't understand a concept, I first ask it to explain the thing to me, and how it improves the project. Then normally I ask it to implement it. I actually never let it implement anything without understanding it myself — that's actually my practice.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: For this, most of the time, we actually write unit tests. The unit tests are actually written using AI, but most of the time the unit tests are very solid — if they don't pass, we don't merge any code into production.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Now, normally, we don't verify as much. Earlier we used to verify, but now it's kind of like some trust has built up. If the implementation is for production, then of course it has to be verified — and when we do verify, we normally verify through unit tests.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: For us, actually, at first when we started, there wasn't that trust, and we made quite a few mistakes. But now, actually — to be honest, AI makes fewer mistakes compared to human resources. Because, say, the code I normally get from my junior colleague has more mistakes in it than from AI. So, now the code isn't verified as much, since we have a separate model to test the code with unit tests.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: Since development has become a lot easier for people sometimes, people can be very lazy sometimes. I've seen many people in my team who normally trust it blindly, and let the model assume the business logic and stuff. So what happens then? Then the models or coding agents end up over-engineering a lot based on assumptions — things that actually don't turn out to be cost-effective later, or break the business model, and everything. And this is very easy to overlook, even for seasoned developers. Because a kind of trust has been built up — which has happened to me too, and to everyone — with AI tools that make fewer mistakes, but do still make mistakes. This should always be kept in mind. So I think, as time goes on, people will lose

sight of this kind of mistake more — let the AI run the whole thing with a proper planning, everything.

P2:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Currently, what's being used is Claude and ChatGPT. Claude is used as the paid one; ChatGPT is the free one. And sometimes use Robo too — the Microsoft one — that's also used as paid. Besides that, DeepSeek is used sometimes too.

Interviewer: Okay, alright. So it's both Generative AI Plus and Agentic AI roughly, both?

Participant: Yes

Interviewer: Okay, did you use it specifically for coding work?

Participant: Mainly, what's used is for documentation work, and for coding work only sometimes — if there's some coding part within that documentation.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: First of all, in coding, what's done is that there are some basic instructions given there. So, following those instructions, I explain to it that this task needs to be done. In the same way, for prompts and documentation, there are usually some visualization elements — say, there's a template, or sometimes a lot of pictures are given, some rough handwriting is given. Along with those, I give a rough prompt saying, 'Based on this description, make this for me' — or sometimes there's already a document shared with it, in its context, and based on that, I say, 'Based on this, I need these parts of this document.'

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Let me say both sides. There are a lot of cases — say, in test case generation, there might be an enhancement or a feature given to me, and I ask it to make some test cases. In a positive way, it creates things — corner cases for these things that I might not have even thought of, it provides those. But in some places, it also gives some extra things that are actually very unnecessary, or it thinks about it and recommends, 'Do you want this thing?' If I say yes — but my thought was that it should only do what was within the prompt I gave — it turns out it adds something extra beyond that too, which I later have to remove.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: This depends. Suppose the prompt is a bit long — then it might ask a question. If it's very short, say I wrote just two lines, then it just goes ahead and does it, saying, 'If this isn't right, there are two versions — you choose which one you want.'

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: The main thing is reading — I read the thing first. In coding, it's about running it — from running it, I actually make sure whether the task fulfills what I want first, then I move ahead saying, okay, this is actually fine.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yes, this happens — it happens sometimes. What kind of explanation — let's say, recently we were working on a documentation, making an SRS. SRS means how the thing will be — meaning the client didn't explain anything to us, so what we did was make one up on our own. Later, when the client gave feedback, that's when the developer remembered the corner case — 'oh, this could also be done this way.' Then he said, 'Okay, alright' — let's say that thing was called Vehicle Management. In Vehicle Management, vehicles are organized according to their number plates. We didn't know that government vehicles have a separate registration — that the vehicle isn't registered under a specific person's name, but under a specific position's/title's name. We had assumed it would be under the person's name. Later we found out that if that vehicle is ever in a case or gets damaged, the repair goes through under that position's name, not the individual's. That kind of thing.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: What I do first is — if I don't know anything at all, if it's very specific, I go keyword by keyword myself, and ask for a definition. I ask it to explain in one or two lines. Then I arrange the sentence accordingly.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: It's actually hard to say, because it's often seen that what counts as 'polished' is different for different people. Say I think, 'the output is coming out fine, this is okay' — for some people time matters, for some, status matters. So often what happens is the output comes to me and I think, okay, this is fine, the thing is done. But it turns out that when I take it to another server — say it was on a training server and I move it to production — it becomes a bit slow there. Then this thing isn't useful to me anymore, or I feel like, no, this isn't going to work.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Okay, basically what I have is that the requirement is written down somewhere — it told me, say, that two plus two must equal four and be shown. So the requirement is in my head. So, however it comes out, first I check whether it matches the requirement. It's the same if I talk about documentation — like SRS, test cases, or a project management plan — these three are three completely different things. Since I know the definitions, I first check whether the thing is correct definition-wise, then I actually move ahead from there.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed over time?

Participant: This actually depends — it depends on what the actual requirement is from the person I'm working for. Say I think about it the way I think it should be done, and I give it that way — I might leave the output like that and stop there. But the client — or whoever I'm working for — says, no, it would have been better if this were done here. Then, based on that, I have to rework the things.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: First of all, what I feel is that the biggest infrastructure-related problem is token burn. So say I have 2000 tokens. Now, for the code I want to implement with those 2000 tokens, the result might not come out the way I want. So I actually have to keep the structural aspect in mind for that, because developers still exist — not everyone can become a developer. So it's seen that if the infrastructure/structure is written out by the person themselves beforehand, where they exactly understand what needs to be done, then the token-burn issue actually reduces completely. Plus, they can also get their exact result out.

P3:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Recently, speaking of paid tools, only Claude and Codex are being used. And among the free models, quite a lot have been tried — DeepSeek, plus Kimi, there was a free version, and Open Code too, that was also tried roughly.

Interviewer: Okay, alright. So you use agentic tools more?

Participant: Yes, agentic tools mostly.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: So I mostly keep the planning in my own hands. By that I mean, I actually first have the AI do the planning itself at the start. Then, until that plan fits with the plan I have in mind, I keep checking the plan repeatedly and having it corrected. So when I feel that — no, this is okay, from every angle: the coding structure, and then proper security is maintained — if everything seems fine, then I hand that thing over to the agent.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? And how did you respond to those additional changes?

Participant: It happens most of the time. Many times it's seen that sometimes it works well, and sometimes it works badly. How it works well — many times I can't think of many corner cases, or I haven't thought of certain corner cases, which usually come up after testing, following the normal flow. So sometimes AI can think of those in advance. So then it's good — suppose something was going to happen, then it can prevent that. But sometimes this becomes a hassle too, because some of the changes it makes, I feel, aren't needed. When it does those, mostly what I do is tell it to revert those things.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: Most of the time, it does ask questions. Since I mostly use Codex and Claude, both of their agentic tools, or the CLI — they mostly ask, before any planning or before any execution, they give options to choose from, like 'is it this or that' — and then based on whichever I choose as my answer, it responds accordingly.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: How do I understand — as I said at the start, actually after planning, when I feel that it's able to carry out the plan properly, that's when I understand that it has understood the task that I'm actually asking for.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Currently, actually, no such scenario has come up yet where I've had to change the code because of that.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: No, my choice is to first know that thing myself. There are some reasons for this — if I hand it over directly, AI will be able to do it. Now, mostly these agentic tools, the good models that exist, can perform quite well, so there's actually no doubt about that. But the thing is, at the end of the day, when the codebase gets bigger or some issue gets created, then it'll be seen that AI keeps going back and forth repeatedly. So that's why, if I have the concept clear beforehand, then whatever kind of issue comes up, I'll be able to fix it easily.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: By specific points, I mean I look at a few things. So if there's an existing project, first I check whether the folder structure or naming conventions that I use are being maintained by the AI or not. Then, whether it's able to maintain the style in which I code. And also, whether it's able to execute properly according to the way the plan was made. If these are all fine, then it's counted as roughly acceptable code.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: If I talk about verification — well, the thing is, I go to another AI and tell it to cross-check whether this is okay or not. After that, if I feel like it's fine, then I accept it. Otherwise, I do some more research for a while, and then proceed.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: Mostly no, not in the same way. Because there are definitely some differences — human-written code and AI-written code aren't actually the same. So the verification for both is also different. When AI writes a piece of code, I mostly try to look through the whole code myself first, most of the time, to check whether the code is actually correct, or whether it's doing what I actually want — then I understand whether it's fine or not. And when it's mostly human-written code that comes in currently, I give that to AI to verify again. Then if AI says there are some flaws or issues, then I check those things myself to see if there's really an issue there, and then I scan through the whole code myself.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: Okay, mostly what I've felt is two things — one is scalability, and the other is security. If these two things are overlooked, there's trouble. At some point, it's seen that AI-generated code tends to be very clumsy, and scattered, this kind of thing. Now, if it can't maintain a proper clean code structure, then that can really become a good problem in the future, because a codebase is never the same — it keeps growing day after day, if the project is more or

less established. So in that case, this can become a problem. And security is a major thing — this also can't be overlooked. Another thing is, I feel that even though it can do everything, its UI/UX still isn't that much better. Whatever the case, human creativity is still needed on that side.

P4:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I currently use Google Antigravity at my workplace, and I use the enterprise version of the tool.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: Well, currently it's kind of company policy to initially start planning with the AI tool. So it's used from the get-go basically.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do, and what did it change and how did you respond to those additional changes?

Participant: Sure. There's this thing we're doing where we are distributing a certain part of the application. So basically the batch app runs on a single instance and we're trying to distribute it. So I asked it to create a plan for the possible architecture of distributing the application. So instead of just giving me the architecture, it also proposed some changes to the front end code which I didn't ask it to do. And when it did that, a feature of Antigravity is that it provides a plan before executing. So I saw that it created changes, it made changes to the front end as well. So I just reviewed it and told it to remove that change and then told it to execute the plan.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: Well, it's a mixed bag. Sometimes it asks for a clarification, sometimes it just makes stuff up. And when it makes stuff up, I have to manually check it. And usually, when it makes stuff up, I find it in the code when it executed, and then I have to change it manually.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: Okay, so before, when I tell it to execute a task, I have to give it a very very detailed prompt, because otherwise it starts assuming stuff, because as I mentioned earlier, it doesn't always ask for clarification. So basically I have to tell it: this is where you need to do it, this is exactly what you need to do, this is what you need to avoid. It only then does it really give good answers.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yeah, so there was this recent massive project where we had to upgrade the Bootstrap version from 3 to 5 in our application, and we did it using Google Antigravity. But it did a change where, since the change was really big, it wasn't possible for us to review everything as it was a two person team. Then during testing and review we saw that it made this unnecessary change, and we couldn't really explain it. So when that happened, we sort of just admitted that okay, it was AI generated, it wasn't intentional, so we just removed it and then put it to test again.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: No, I ask for explanations, because otherwise if it generates something faulty, I won't know what it did wrong.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: Well, I review it after it generates a change. Usually I generate it then push it to the repo, because it's easier to see the changes, and when I see that it made a degenerate change, I tell it to remove it, or I remove it myself manually.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Well, I try to verify it most of the times. If it's quite simple, I verify it every time. So it depends on the use case. If it's a very simple change, then I just go through the code and see if it works. But if it's something like a change in business logic or something completely new, I have to run it. So when I run it, I take the changes or the edits it made, I deploy the code and then see if it runs properly or it runs as intended. If it doesn't, then I change it again.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: It has somewhat. So I do use AI tools to review the code as well, but I still don't really trust the output of the AI tools yet, so after seeing its reviews, I go through it manually again. So it's kind of changed, but I won't say the change is that drastic — it has made it faster.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: Reliability, blind faith, and the tendency of tools to agree with anything you say. So basically reliability is one of the bigger issues, because AI tools don't always produce reliable code, and if developers don't review their code properly, the issues might pop up way too late and cause problems in production. Blind faith is, well, it's related to the first one, and the tendency of AI tools to agree with anything you say. Basically, if you say something that's not really logical or isn't congruent with the business logic, AI tools tend to agree with you, and it agrees in such a way that the developers may think, "Yeah, I'm right," but in actuality, it's not. So this is one of the issues — it doesn't really refute the developer unless explicitly told so. And most developers don't really cross-check that and don't really practice refuting their own thoughts with the AI tools. Yeah, these are the three issues I think are the biggest ones.

P5:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: The AI tools I currently use are mainly free. Firstly, I don't have any credit card. I mostly use whatever free models are available of open-code. And among the online based or browser-based tools I use are Claude, Gemini, ChatGPT, and DeepSeek. Depending on the requirement, I customize things myself and use whichever one I need at that point.

Interviewer: So you use conversational tools and agentic tools both?

Participant: Yes, conversational and agentic – both types I use. Whenever what is needed.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: I don't completely hand over any task to AI. My workflow is roughly like this: when I have a structured task in front of me, I first brainstorm it myself and bring it into a specific format. There is a detailed instruction set covering what needs to be done, how each thing should work, and so on.
Then, based on that instruction set, I give the task to an agentic AI tool.
If I give an example from my real-life experience, I have done two projects—two Telegram bots. The work of both bots was roughly similar but I have implemented them differently. One was for monitoring the electricity meter at my home, and the other was for scanning the home network. So we can connect them with Bot in two different ways. One way is, Questions will be sent to bot, I will send a question and if the question falls into the queue the answer will come to us. And another one is, no one will ping directly, that it would hit the bot API and fetch.For both of these projects, I gave the agentic AI very clear and detailed instructions. The AI prepared the entire code. Then I went into the code myself and checked where the implementation needed to be changed or edited, made those changes, and then finalised.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change?

Participant: This definitely happens. In fact, sometimes it does a bit too much. Basically, as you know, I work with both journalism and independent work. So in many cases, when the instructions are not clear enough, AI starts adding more information or doing more than what was actually asked.
For example, if I ask it to write something about the current Prime Minister of [country], AI may sometimes give me the name of a previous Prime Minister instead.
Or it may get things like the Prime Minister's name or whether to use "he" or "she" wrong. Then I have to point out to AI that "it will be he or she " or "prime minister will be this not that" .That's one example of a context-related issue.
Another thing I've noticed is that, with a free tool—I don't know if this specifically happens because it's a free tool—it sometimes adds an extra dependency when I am coding, even though

it's not actually needed. Later, I find that the dependency isn't used anywhere else in the code; it's just imported and left there.
Basically, I work with Python. Another thing happens, I don't need a certain type of implementation but it added the implementation still. So, I've faced this kind of issue as well.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: First, I ask the AI to structure my problem to a point where there should be an obvious answer. I want the process to follow a planned approach, and I want to move toward that obvious answer. Within that process, I give the AI some degree of freedom—I let it ask me questions along the way.
In that skill or prompt, I tell it that if it wants to make a decision at any point, it should ask me, "Do you want me to make this decision?" It should let me know before taking that step.
Similarly, if it feels any ambiguity at any stage, or if something is unclear, it should ask me a question at that specific point. So I pre define this early
That's the approach I follow. In that case, I find that the chances of making mistakes are lower.

Interviewer: So it asks for clarification based on your plan and it don't give based on its assumptions.

Participant: Yes. I want to make it work based on my plan.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: In the beginning, when I used to give instructions in a very unstructured way, I didn't use AI that deeply. In those cases, when I wrote the code, I would later forget why something had been implemented in a particular way.
Now, what I do is, whenever I ask AI to write code or have a conversation with it, there is always some reference for that work. I also tell it to follow the existing standards.
For example, if I write a class, I ask it to add a docstring below the class explaining, "This class does this, and these are the responsibilities of its functions." I ask it to document those things.
Similarly, if it finds that a function contains some ambiguity or something particularly complex, I ask it to add a docstring below the function explaining what exactly it does.
I also tell it that during the implementation, it should explain to me in its response why it implemented something in a particular way.
So, this is basically something I've learned from my experience.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: In this case, what I usually do is, suppose there is a concept that I don't understand. When the AI explains it to me, I ask it, “Give me the references you used to explain this to me, numbered 1, 2, 3.”
Then it provides me with the URLs for those references.
I read those references myself. I don't ask the AI to summarize them for me. I read them and try to understand them in my own way, and then compare my understanding with what the AI explained.
If I find any discrepancy, I ask something like, “I found this information, while you found this. It seems like the situation is actually like this, so what should we do now?”
That is how I continue the conversation.
So, before starting any implementation, if there is something I don't know, I first learn about it. Once I understand it, I make sure I have internalized the concept in my own way. Then I tell the AI to move on to the implementation: “Okay, let's implement it this way.”

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: One important thing is that I would say this from both perspectives—journalism and coding—based on what we currently do in our respective professions.
In journalism, most of the raw reports we receive from our reporters are currently AI-generated. So, the first thing we do is give the content a human reading. I personally read the content and note down the important points, usually in brackets, such as what information is present and what needs further attention. Then I have a conversation with the agent or conversational agent about what other related issues, inconsistencies, or missing information might exist around those points. After that, we use the review we receive.
I follow a similar approach with code. I haven't worked on extremely large projects with hundreds of thousands of lines of code. For smaller security scripts or security-related code, I first read and try to understand the specific module or section I'm working with. If it needs polishing, I go through the same process again—I do a human reading, identify what is missing or incorrect, and then give the AI a specific instruction to fix it.

Interviewer: Does it happen that the structure or the process of writing the code or method, maybe it could have been done in a different way? Do you converse regarding this?

Participant:
If I feel that the AI isn't structuring something properly, I can continue the conversation with it. For example, with Python's PEP 8, I can compare the relevant PEP 8 references with what the AI has produced. I can also ask the AI to first verify whether the implementation follows PEP 8. Then, of course, I still validate it based on my own experience. The same principle applies to our news reporting process as well.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: I’ve already answered that earlier. When I take an answer from the AI, I tell it, “You should find authentic references on this topic and give me the links.” Then I personally go

through those references and read them myself. It takes more time, but it reduces the amount of risk.
And this is not only for code. We follow the same approach when dealing with sensitive writing as well. For example, when we are writing something related to policy-making or politics, we carefully organize and review several things. In those cases, it is not just about linguistics; there are also factual aspects and aspects related to human cognition.
So in those situations, we read the references ourselves and use the guided references to verify what is actually happening and whether the information is accurate. And the same goes for the code.
Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?
Participant: I've already explained my whole procedure. Yes, there has been a change from how I used to work before. It's really a process of continuous learning. As AI tools and agents become more integrated into the workflow, there will still be certain areas where human review or human intervention will be necessary. There's no point in simply copying and pasting. You still have to make changes and adapt the output.
And this will differ from person to person. Someone may be satisfied after writing a general report and think, "I wrote it really well." Sorry, since I'm in journalism, I'm focusing more on that side.
But the reality is that someone can also feel very satisfied after writing a piece of code and think, "I've written a really good piece of code; it works fine." But then I might look at it and say, "Brother, it's fine and it's running well, but when this amount of data comes in, your application is going to fall apart." So, that's the point.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: I haven't really worked outside the country in that kind of environment. I've worked in [country], both in the tech industry and in the media. What I've come to realize is that AI or agents are not for people who don't want to think. By that, I mean people who don't want to put in the effort to think. AI is for people who are willing to think and who are used to putting in that effort.
The biggest limitation I've noticed is this: just as you can't use a goat to thresh rice, you can't really use a goat to operate a computer. But yes, if a goat starts typing on a computer, something may come out of it.
We are living in a situation where the goat is essentially the person behind the keyboard. They use AI and think, "Look, it's already building the pages for me. Why do I need to do any of this myself?" So, that's what I've come to think: the people running our businesses are often very intelligent, and ironically, those intelligent people can become the biggest problem when using AI.

Interviewer: Do you think it is a risk that developers are developing with AI without understanding the concept? Just because it's working, they publish it. Do you think it's a risk?

Participant: When it comes to development, whether we do it with AI or with humans, if the code works and test works ,we push it to the production. Our culture is already risk.

P6:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Currently, I use Codex, and I use Claude both are paid versions.

Interviewer: Okay, do you use conversational tools more, or do you use agentic tools?

Participant: Agentic

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: First of all, this will be a bit long. I mean, first I actually plan the thing. After planning, I review the plan very thoroughly. When I feel that the thing is finalized, I revise it once using both ChatGPT and Claude. After that, I actually make something like a roadmap. Then I divide it phase-wise, slot by slot, and that's basically what I follow currently.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Yes, what happens is, when I'm building some application, and my instruction set for it isn't very detailed, or I haven't gotten enough details from the client either — at that time, when the instructions are a bit less, if I don't specify things more, it tends to do a bit more than asked. But if I state things very specifically, then this issue doesn't happen. Normally I've faced this multiple times — what happens is, when I'm having an AI tool do something — I'm a backend developer, so my weak point is normally the frontend. So for the frontend, I'm not able to give instructions as clearly as a frontend developer could. So in that case, often — regarding design colors or whatever — when it doesn't come out the way I wanted, I revert the whole design, and then I take help from it on how I could write the prompt in a better way. In some cases, I also generate a mockup for this. After looking at the mockup, once I feel the mockup is right, I generate the instructions from the mockup and give those to it.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: Normally, if I tell it that I'm doing this kind of task, I want to do this — at the initial level, what I've observed is, with Codex and with ChatGPT too, it actually picks one standard approach and moves ahead with it. But with Claude, in places where I'm looking at business logic, fixing architecture, meaning programmatic areas, Claude asks a bit more questions there, and Claude's specifications actually turn out well. So in that case, Claude asks repeatedly, but Codex, it seems, just moves ahead on its own in a certain way. There might be a slight difference, but this is what I've experienced.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: First of all, this is the process I currently follow. As I mentioned a moment ago, before doing any project, I always maintain a 'docs' folder very carefully, and inside that docs folder, I've actually done quite a lot of research on how docs folders should be structured in a

project, and how I'll execute things day by day. In these docs folders, my architecture is fixed — even for the frontend, what kind of colors, what kind of theme, and so on — I define all of that in advance. And normally, when it produces the output, the AI does the testing, but for the output itself, I try to understand it a bit by doing self-testing, to check whether it actually delivered things properly or not.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yes, what happens is, often when work is done using AI, the people who don't review the code properly end up skipping over things. It creates a lot of files at once, generates a lot of things that are actually outside that developer's usual habits or style. So this does happen from time to time, because when a somewhat big feature is being built, things kind of get lost in the process. Specifically, there are normally two kinds of issues here. While I'm working, some business logic comes up. At that point — in many of our existing applications where we use AI, our business documents aren't defined very well. So there, when developers go to work — many developers — they often end up changing the main business logic itself, without realizing it.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: Normally, if I'm doing any project, or trying to do something — earlier on, I used to say, 'give me a good idea,' and then just have it implement that good idea, meaning I'd leave the whole thing up to it and just want the code implemented. What happened in my case was that, ultimately, the result would end up quite far from the way I was actually thinking about it. For example, recently I was building a simple app for fare calculation. The way I actually think and work, it didn't come out that way — it wasn't what I expected. So now the approach I follow is: when I'm going to work in a certain domain, I first study that domain properly and try to look at existing products in it. After that, I make my own written set of notes, and then work within AI based on that. I don't let it explore freely, like 'I'll just do it this way, you bring in all the good stuff and merge it here' — because then it's not really controllable. If there's a project with documentation maintained, I ask it whether the existing documentation and the actual code features are in sync or not. If they're not in sync, I ask it to sync them first, and before syncing, I confirm whether the code is correct or the document is correct. If that can be confirmed, fine — if not, I sync the code and document first, then start working. But if it's an old project where there's no documentation at all, I actually produce the documentation properly first, and then want to work on it. Because there are some 'garbage' aspects to consider — what to do, what not to do, where I can touch things and where I can't.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: In this case, what I do is — since I'm a backend developer, when something changes on the backend, normally I maintain one folder, one repo, for both backend and frontend. This makes things a bit convenient. When I go to the frontend, I understand the frontend part mainly by looking at the outcome and some initial things, but for the frontend structure, I fix a structure early on that it should follow, and it follows that. For the backend, I

check whether it aligns with the existing architecture — whether I work this way, and if I do, then I go along with that. But if the AI doesn't maintain that, I tell it to follow this structure instead.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Yes, I do verify whether it's correct. What often happens is, when I take a suggestion from ChatGPT or somewhere else, the more specific I am, the better the answer I get. Whenever I say anything to ChatGPT, I explain the full context — that I'm developing an application for these people, in this place, to solve this problem. And for verifying, I actually cross-verify — I get something from ChatGPT, I put it into Claude as well, and check it there. So if it checks out from three directions, I accept the thing.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: In terms of change in how I verify AI output — for the end output, I do check it by visually testing it. But something I still haven't been able to adapt for verifying output is this: I maintain test cases very well, because right now everyone's saying AI will give you the code, but you can't just pass it without reading the code. But that's actually not being maintained, and it won't be, because AI does in five minutes what would take a human an hour to read through. So, to control that, I try to design two things very well — test cases and documentation — so that things don't break down in some way. I do review some places in the code, but I don't review everywhere now. For the parts where I do review, first I check what will actually differ. As a backend developer, I check whether the database is set up the way I've thought of it and the way I want to keep it, future-proof, and I confirm that. And I check whether the structure is kept the way it should be — my folder structure, or any design-related matter — and authorship-related things too. I look at a few particular spots very specifically, and try to confirm that the whole project follows the same kind of structure, whatever the code looks like. Say, this module is done this way and that module is done a different way — I don't allow that; if it happens, I ask for it to be refactored.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: What can be overlooked is mainly this — code ownership, practically speaking, basically belongs to no one anymore, for those who use AI very heavily. Because that's not really practically possible anymore. What used to be the case was, a developer would be fully confident about what's happening where and how. But now, that's not the case anymore, or won't be. So now, a developer has to review another developer's work again, and also has to review the AI's work again. This is actually a major issue. Another issue is that if you can't check the prompt very carefully, AI often uses cutting-edge technology that's actually not suitable for the situation. This kind of problem can happen when it comes to choosing technology.

P7:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: I use a paid version of Claude. I mainly use it as a coding agent. I sometimes use it conversationally when I need help with decision-making.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: I generally use the agent first to analyze a task. If I find that doing the work myself will take too much time compared with the amount of guidance I would need to give the agent, I decide to let the agent handle it. Later I review it.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: No, it didn’t happen. It didn't happen with Claude. But when I used the free tool Copilot that time changed some stuff. When Copilot did this, I lost trust and reliability.The same kind of problem happened several times. It was changing core file even though I had explicitly told it not to change. That’s why I shifted to Claude.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: What Claude does is – it asks me questions.It usually asks me two or three questions. It also recommends one of the available answers, so I can choose an option or write my own custom response. In my experience, it generally seeks clarification instead of silently continuing with an assumption.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: I maintain a Markdown file containing detailed project instructions, including how the project should be structured and the guidelines the agent must follow. When I give it a prompt, it reads that file before doing the rest of the work. In that type of work, it is reliable. And I check every time — how many files have changed. I checked these things. I plan first ; core planning stays in claude.md

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: This has happened, particularly when reviewing code that used an unfamiliar technique. If I still do not understand a term or implementation, I either question the agent or research it online. In one case, the AI implemented a state-management mutation in a way I did not initially understand. I asked it why it had done that and what the benefits were, and it then explained the reason.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: For an unfamiliar project, I first share the project context with the AI so that it can understand the codebase. I ask how the assigned task could be done and why that approach is appropriate. When it proposes a change, I ask it to break the reasoning down and explain why the code is structured that way. I explicitly tell it not to implement anything until I provide final instructions, so I understand the concept before allowing it to proceed.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: I check it manually often. I check if the structure is correct. I first inspect the result visually, then review the code structure. I check whether the change could create side effects and whether it modified any other files unexpectedly.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: I do not usually perform a separate verification for every ordinary conversational answer. If an answer creates uncertainty, however, I ask another AI agent the same question and compare the responses. Suppose, after getting the answer from ChatGPT, I ask DeepSeek again to check if it is similar. Looking into 2-3 agents helps with the idea.

Interviewer: As you've become more familiar or comfortable with AI tools, has the way you verify their output changed?

Participant: Day by day, we are also learning how to check its output. It (AI) makes changes in a lot of places. In a small amount of time, it does a lot. So, manually even for reviewing the report, we had to become smart gradually. I am going on with different approaches for that, for example : Instead of looking at it this way, perhaps we could approach it from a different perspective. We are still learning.

Interviewer: Is the process different from reviewing a colleague's work?

Participant: Reviewing the same way becomes difficult. Firstly, AI generated code is more or less trust worthy.I use an extra high reasoning setting (effort settings high), which gives me some

confidence in the result. When I make changes to a file, I check if the states are correct as I work with TypeScript. So I do type-checking. I don’t need to check the markup, as I’m already fairly confident that it won’t make mistakes in this regard. If I find a problem, I ask whether an alternative implementation would be better.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you've personally experienced that you think developers may overlook when using AI?

Participant: The biggest risk is losing the desire to learn new things. When developers encounter an unfamiliar term, they may let the AI finish the task and simply move on. If they skip the underlying concept, they do not learn and their own development eventually stops. Instead, they should ask the AI for a deeper explanation and continue learning rather than allowing the convenience of AI to make them reluctant to learn. Now because of AI, we don’t learn anything.

P8:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I use Claude Code and Cursor. Both are paid.

Interviewer: Okay, do you use conversational tools, where you give it specific code or questions, or do you use agentic tools that work with access to a large part of your codebase and can make changes?

Participant: I use both. Both are needed.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: Could you say that again?

Interviewer: At what point do you decide to hand a task over to the AI instead of doing it yourself?

Participant: Now that AI has improved quite well — earlier, most of the time I had to do the work by hand myself. Now, especially with a model like Claude, it can do all the work if you explain it to it.

Interviewer: I mean, when do you decide that, okay, I won't do this task myself?

Participant: When I see that it's doing the task properly, then I hand it over. I do the planning by myself.

Interviewer: Okay, so once the planning phase is done, then you hand it over to the AI?

Participant: Yes

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Actually, it's like — basically, I generated, like, test cases with AI. Earlier, what would this AI do? It would give steps, give scope, give exactly what was asked, right? Now it mainly gives the base, and it also asks questions, and it also recommends features for you. Sometimes it develops some extra things; some developers don’t like that.

Interviewer: Okay, okay. I mean, has it ever happened that, say, you were writing tests — test code — and it touched places where you didn't need it to, places where you had told it not to touch or where it wasn't necessary — it touched them without permission?

Participant: No, AI asks permission

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: If it's a good model, it asks. If it's a smaller model, it starts making things up.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: Let me give an example of how I do this. Normally, with a good model — the kind that has to take in more context, like Opus 5— I make a plan with it. Then I tell it to make the plan in such a way, and to break it down in such a way, that a smaller model — like cursor 4.6 grok model — can understand it and implement it properly. Then I do it with small touches: it does the first part, then I have that checked , first check it with grok if it's passed , then I also check it with Claude to see whether it understood the plan or not.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: This happens if I don't do things properly, step by step, or if I'm in a hurry because of a deadline — then I fall into this situation.

Interviewer: Okay, okay. Can you tell us what happened when an incident like this occurred with you?

Participant: Let me tell you. My product manager wanted us to quickly build an AI tool in Playwright. In one of our projects, logos were going to change, so the tool was meant to detect all the logos and generate a report. Now, there was no time — so what it did was gave a lot of bugs , not like that gave the obvious one. So the result wasn't what I wanted.

Interviewer: Okay, I mean, did you have trouble understanding the code it wrote for Playwright — how the code works? That's really the question: it did something, and later you felt something needed to be changed here, but you couldn't understand how the code works?

Participant: No. If I fall into such a situation, I'll ask the AI for an explanation.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: No — I first understand the concept, then I do the implementation.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: If the output is within my expectations, then it's acceptable.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Normally, I use multiple models and see whether the other one is giving the same answer or not. If it's the same, then I think it is correct.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: Pardon?

Interviewer: I mean, suppose you've gained experience using AI, and now you use it much more comfortably than before, right? So the way you verify AI's output — is the review method still the same as before, when a coworker wrote some code, and you reviewed it in that way, or is it different now?

Participant: I'd say the straight answer is that it saves a lot of time. But at the end of the day, I feel it's the same as before. Earlier, tasks that were simple still took time to do; could have done it more calmly. Now there's more work but less time, and colleagues using the AI also sometimes makes the task complex, so you again have to understand the task carefully with AI and do it. So I feel that nothing has changed — the review process is still the same.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: I'd say that if you work slowly, the chance of missing something is very low, because you're reviewing properly. But normally we can't do that, because time is fixed. So at the end of the day, you have to trust the AI.

Interviewer: So it turns out that, many times because of time constraints, you don't review the code properly, and just leave it because the code is working, right?

Participant: It's not just me — I think everyone is doing it.

Interviewer: Yes. Is this what you consider the big risk?

Participant: Yes. This is certainly a matter of concern.

P9:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I have a ChatGPT subscription. I use the free version of Gemini as much as possible.
I use them for a lot of tasks. First, I use Claude Code as a coding agent. Different tasks, bugs, feature enhancements, planning mode for planning, and discussion—I majorly do almost everything through Claude Code. When I need frontend asset generation, or when I get stuck on a problem with Claude Code and need a different point of view, I switch to ChatGPT. If I need to generate an image, I also use ChatGPT, because it works well for image generation. Basically, when I get stuck, ChatGPT is my fallback. The current version of ChatGPT is also quite good. So, if I need a different perspective from Claude Code on any coding problem, I use ChatGPT. I use Gemini when I need something specific related to frontend design—for example, a large SVG or some other large asset. When I cannot easily find it even through Google Search, I work with Gemini. Gemini is also good at some specific tasks—for example, [local language] translation, file translation, translating JSON files, and large-context tasks. So, I use Gemini for these kinds of things. Besides that, I use some other tools as well. For example, sometimes I buy credits and use Seedance. I have used different Seedance models, such as Seedance 2 and Seedance 2 Mini, for video generation. They generate good videos. These are also used for landing page content or the hero section. Currently, landing pages in frontend usually have some kind of animation, where an element in the hero section or background image moves. Sometimes there can also be moving elements on the login page. Because of this, I have developed a workflow. First, I generate an image using ChatGPT Images. Then I animate that image through the sedance model and ask it to create a small, abstract-type video. What kind of animation or video I create depends on the project. In some projects, GIF is used, in some projects SVG is used, and in some projects video is used. So, I select the asset based on the specific requirements and the design of the project.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: So, first, when you go to do the task, you have to understand the task—how feasible it is, how impactful it is, and how much change we need to make. Because I need to understand the architectural decision and the higher-level things. Otherwise I can't explain what I need. I need technical understanding. I mean, I can't understand the technical understanding. Then I have to go to the high level of my code. So, I can make some changes in both my files, sections, and modules. So, when the whole thing is planned and a rough idea comes up, I know the requirement. I know what I want, I know my findings, I know my result—I have a rough idea. Then finally I say that these are what I want to do and this is how I want to do it. Now you can do it. Then finally I see what it actually did and whether it logically makes sense, whether it coded it correctly. So, whether the coding is correct. Now I have to validate it. At the end of the day, the feature is mine. Most of the time, when it goes through validation, an edge case will come up that they hadn’t thought of either. Maybe it's a very complex scenario. Or maybe because it could be a nuanced case. Then you have to go back and forth again to get it fixed.

Then you have to go back and forth to fix it. And in the case of UI, it's a little easier because you can see a visual. But the logic usually isn't there. So, here it's case-based. I mean, a lot of times it happens that when you have to do one thing, you have to do a lot of things. Or you have to give a lot of things out of scope.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: What happened last week was that there was a form validation in jQuery. There were three specific conditions that I mentioned for the form, but it was still allowing the form even when the conditions weren't met, or even when only one of the conditions was met. Basically, it couldn't pinpoint that particular issue. At first, I thought maybe I had given it the wrong instructions. Or maybe there was an issue with an enum. Or maybe I had mentioned a specific enum, but that enum was actually something different. Later, I found out that it actually wasn't able to find the query and the form. It was having trouble figuring out where it should stop and watch. When a form keeps getting submitted on the frontend, you can handle it in two ways. One is to check at the final submission, and the other is to check per type. As far as I remember, it was causing an issue in one of those places; I don't remember exactly what happened. But then again, I had to go back and forth with it for four prompts. I had to explain the scenario to it properly. Then it finally managed to fix the form validation. It was really annoying. It wasn't difficult at all. It should have been very easy. But it took a few tries. It didn't work with a one-shot prompt; it couldn't do it. This does happen from time to time. I mean, it's not like I give it a prompt every time and it just gets done.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: It's getting a lot better gradually over time. It's far better than before now. Claude's models are especially really good—the Sonnet ones, the Opus ones. Opus 5 is just exceptionally good, I guess. It actually stops me on its own and says, "Okay, you've missed this thing. According to the code logic, this shouldn't actually happen." I faced something like this last week. I told it to do X, Y, and Z, but it said, "I can do X, but for Y, this thing is missing. And for Z, this is required. That's not there. This is unclear." It stopped on its own and told me about it first instead of just going ahead and doing it. But then again, there are times when it just does things on its own. Then I realize, "Oh no, this wasn't supposed to happen like this. What just happened?" And then I have to fix it again. So it actually goes both ways. Recently, though, I've noticed that it's getting pretty good at understanding when it should stop and ask instead of randomly going ahead and doing everything on its own.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant:I don't really know how effectively I'm communicating with it. Since I'm getting the output I want, I guess it's working pretty well. Basically, I give it a list in bullet points, almost like how a senior developer would communicate with a junior developer. Like: "First, do this. Then do this. Then this. Then do this. Finally, this should happen, meaning you need to validate that everything is correct." Then I take that same list of things I gave it and validate them again. I look at the code briefly—just a rough overview. Not line by line. Just a quick

glance through: “This should do what it’s supposed to do.” After that, I play around with the UI a little. If there’s a UI change, I check the UI change. If there’s no UI change, then there’s nothing else to validate there. In that case, I have to go through the code logic a bit more carefully. But if there is a UI change, then that itself gives me a point of validation. So, for validation, I do a rough overview of the code—not line by line, not word by word. I just check things like: it should be in this file, it should be in this function, this logic should be handled here, etc. After that, there should finally be the API call or the UI change, or whatever the expected outcome is.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: I mean, now you can actually tell when AI is involved. AI itself isn’t really the problem. The problem is understanding why it did something. Comments often aren’t written properly. A lot of people don’t write comments at all, whether it’s AI-generated code or self-written code. That’s when the problem starts. I mean, there’s no problem understanding the code itself. But if there’s no comment explaining why the code was written this way, then, honestly, only God knows. And whoever wrote it knows.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: So recently, I worked on something like this—a project, basically. I used Python and FastAPI for the backend and deployed it on a VPS, along with the database. I used PostgreSQL for the database. But I don’t really have a strong understanding of those things. Then the frontend was built with Astro, and I don’t have that much knowledge of Astro either. So I told it whatever basic concepts I do have—things about databases, primary keys, foreign keys, how things are supposed to work, why a query might be taking too long, or how I would normally approach something in React on the frontend. Based on that, it built the whole thing for me in Astro. It even deployed the whole thing using the API. The problem is, I can’t really do a proper code review there. I don’t have the coding knowledge or familiarity with that stack to know exactly how I would implement that specific part of React in Astro. What I can check is whether the end result is behaving the way I expect it to—whether it reflects what I wanted, whether it works the way it would in React, and so on. But I can’t really review the code itself because I don’t have the knowledge in that area. At the same time, I actually want to learn from it as well. Like, “Why did you do it this way? How did you do it? Teach me and explain it to me,” so that logically, it makes sense to me. Because over the years, I’ve developed a general understanding of backend and frontend—how things are supposed to work and how they should be structured. That’s basically what I’m relying on here.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: Like I said earlier, the first thing I do is a rough overview during the code review. That’s when a lot of things already become obvious. For example, this model’s code isn’t supposed to be in this section. But it has put everything into one file. It hasn’t even defined the service type properly, and the interface isn’t properly declared either. These things immediately

stand out when I go through the code at a high level. These are basically structural things. This should be in this file, this should be in that section—that's part of the structure, right? Say we have several sections or modules. Four modules belong in one place, two modules belong somewhere else, and another module belongs somewhere else. If it creates a separate common file for ten modules when that isn't part of the coding structure of my repository, I'm not going to follow that, right? I would tell it, "Look, this is how the structure is maintained in this repository. Follow that." Because if its code increases the complexity of my codebase, that's a problem. If the code isn't human-readable, that's also a problem. If someone else looks at its code and can't understand it, that's a problem too, right? The AI might be able to read it and figure it out eventually, but humans have to understand it too. It needs to be easy to read. And making code readable isn't difficult. You can explain the logic later if necessary. Comments also need to be written properly, and unnecessary comments shouldn't be there. They should be precise and short—only the ones that are actually needed. Those are all things you have to check during code review. So code review is basically the first phase: making sure everything looks right. It smells good, it sounds good, everything looks okay. Then comes functionality—whether it actually works. I have to test it. I have to test the edge cases and find them myself. There might be some complicated or tricky scenario that I didn't mention, and the AI might miss it. Because otherwise, there's really no way around it. So I can't just trust it. The AI agent is basically my junior developer. It has the ability of a senior developer, but it's doing the work of a junior developer. It can make mistakes. I can make mistakes too. My guidance could be wrong, or it could give me a bad answer for some reason, or it could hallucinate for some reason. There are so many things that can go wrong.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: If I talk about a task where, say, I have a complex table, and inside that table, each row has a complex accordion section where a lot of data mapping is happening—I'm getting raw data from the backend in a specific format. Then I'm doing a lot of mapping and aggregation to display the data. In cases like that, I need to go through the loops and understand how the data is being generated, which function is exporting what, which helper functions are being called, which utility functions are involved, and so on. Say there's some rounding involved. Say there's a lot of hidden logic going on. A lot of weird things are happening in weird ways. I first need to understand all of that—what should happen after what, and what the expected flow should be. But I also need to understand how it should happen, right? I need to have that basic idea. I know what I want from it and how I want the functionality to work. Based on that, I should be able to understand that, for example, maybe this functionality requires the loop to run three times. Not necessarily three times specifically—let's say that's what the functionality requires. Or maybe I see that it's looping six times. Then I notice that it's rounding something off to five decimal places, but the input box in my UI doesn't even have enough space to enter such a large number with five decimal places. I can immediately understand that something is wrong. My business requirement was to round it to one decimal place. These things are very specific to the particular case. They're not generic things. So first of all, you need to have the knowledge—the basic knowledge required to validate these things. Once you have that, you can use the AI to do the work and move much faster. You can point things out properly and avoid giving it the wrong task. But if you don't have the capability to validate what it has done, then there's really no point

in deploying that code. Otherwise, you're just giving it code without understanding it, pushing it, and deploying it. There's no concrete answer as to whether the code is actually correct.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: It's not really different. Earlier, we had to rigorously go through a colleague's code, and now we have to rigorously go through AI-generated code as well. Because at the end of the day, if the mindset becomes, "AI wrote it, so it must be good," then there's a problem. Just like we used to rigorously test a colleague's code, if I could leave even one useful comment during a PR review, I'd be like, "Yay!"—I found something and commented on it, and now that comment will be reviewed. AI-generated code is pretty much the same. I would actually say it's even more critical. Because if we miss something in AI-generated code, it can make its way all the way to production. And when that comes back as a production issue, that becomes a much bigger problem. At that point, we're the ones to blame. No one else. It's a very serious matter. If anything, we need to pay even more attention to it now. If people don't take it seriously and just think, "AI already wrote it, so I'll just merge it," and treat it like a one-shot prompt—"Yay!"—then that's simply not going to work.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: AI keeps giving you unnecessary features and unnecessary improvements, over and over again, right? If you keep telling it, "I don't have anything to do today, tell me things to do," it will give you 10 or 20 tasks. But in real life, in a product, not everyone uses every feature. 90 percent of the time, you'll see that nobody actually uses them. People use maybe 10 percent of the features or tasks. So we should be putting all our resources, effort, and time into that 10 percent. If you keep asking AI, it will keep telling you things like, "Yeah, improve performance, move to microservices, turn the whole world upside down." That's not something you should actually be doing. That becomes a big problem. We should be using AI in a useful way. The same applies to our engineers. Just because AI is available, it doesn't mean we have to do everything it suggests. That mindset isn't necessary. We need to understand which features are actually valid and which ones are actually needed. AI also often removes or replaces existing things. In the name of modernization, it changes everything. But if something is already working, there's no reason to touch it. Because that's production code.

P10:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Currently, for code-specific work, I use Claude Code, It isn't free and I use the paid subscription. For chat-based things, I mostly use ChatGPT—especially for normal, day-to-day, basic things. But if it's something analytical, logical problem-solving, or coding-related, I rely on Claude most of the time. Whether it's understanding a research paper, understanding a model architecture, or doing coding work, I mostly rely on Claude. For understanding research papers, I use Claude's website. For code, I use Claude Code, for development-related tasks, I use Claude Code.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: Mostly, I use it during debugging or optimization. Initially, I write the code myself. Then, once the code is done, if I feel that my implementation may not be efficient, I use Claude Code to verify it or ask it to optimize the code. I also use it during debugging, which is mostly during testing. For example, I have a problem statement and I have implemented a solution in a particular file. I can ask Claude Code whether my solution passes all the test cases or fulfills all the given conditions. Another situation is when the task is very lengthy or tedious—for example, during data analysis. If the task requires a lot of thinking and I don't have enough time to spend on that thinking, I use Claude Code. Sometimes I may need to crawl through multiple folders—for example, my dataset might be in one folder and my solution code in another. In those cases, I use Claude Code as well. Basically, when I need a quick solution and I cannot afford the time required to think through everything myself, that's when I use it. If I feel that a particular part needs optimization or verification, then I go to Claude. I don't use it before that.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Sometimes the agent starts doing things that are completely outside the scope of what I asked for. For example, if I ask it to make one particular thing red, it might end up changing the entire background to green and yellow—basically doing things beyond the scope of my request. This also happens with Claude's website interface. I may ask it to write a simple function, but instead of just writing that function, it might create an entire class, read files, write to files, and add a lot of other things that I never asked for. Those things are unnecessary, redundant, and not helpful for my use case. When that happens, I usually discard the entire solution and try to implement it myself from scratch. Then, after I have implemented it myself, I give my implementation to Claude and ask whether it is correct or whether there are any issues with it. I feel that when I ask an LLM to build an entire piece of code from scratch, it tends to create a lot of unnecessary things. Instead, if I give it a specific reference implementation and ask

whether it is correct or buggy, it can maintain my coding style while making improvements. In that case, I get much less redundant code.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: When I use Claude Code, it usually asks me for clarification whenever something is unclear. If the requirement is ambiguous, it tries to understand all the requirements first and then writes the code. We often see Claude asking multiple questions and requesting clarification in these situations before actually implementing anything.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: What I do is basically explain the entire situation to it. I have been given this task, this is what needs to be done, this is the domain, this is the dataset I have, this is my input, and this is my expected output. I try to provide as much context as possible and explain everything as clearly as I can. If there are additional conditions—for example, something must or must not happen—I try to specify those very clearly as well. Sometimes I may have to work on something that is outside my area of knowledge. In that case, I tell it directly that I don't have enough knowledge about that particular topic. Then I ask it what I need to understand before I can work on it. For example, I might ask: "If I want to understand this research paper, what knowledge do I need to have first?" I ask it to generate a list of the topics or prerequisites I need to understand. Then, if possible, I also ask it to provide sources—for example, which blogs, articles, or other resources I should read to understand those topics. After understanding those prerequisites, I start working on the task. While working, I continue asking questions whenever I encounter something I don't understand—for example, "Why is this happening?" or "Why is this condition necessary?" I also question the code it generates. If something doesn't make sense to me, I ask whether it is actually supposed to work that way. Basically, I try to make sure that the implementation is within the level of knowledge I currently have about that domain, so that I can actually understand the code being generated.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: When I write code completely by myself—sometimes badly documented or without comments—I've personally experienced situations where, after some time, I come back to my own code and don't remember why I implemented something in a particular way. With AI-generated code, however, when we copy-paste or verify the code, it often already contains a lot of comments explaining why something is being done in a particular way. As long as I don't remove those comments, I usually don't have much of a problem because I can go back and understand the reasoning by reading them. But if I remove those comments after copying the

code, then yes, sometimes I forget why a particular condition or implementation was necessary. In that situation, I may have to go back to the AI and ask again why something was implemented in that way.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: The format I usually follow is that I first tell it how much I already know and how much I understand about the topic. Then, for completing the task, I ask it to describe or generate a list of the prior knowledge I need. I ask it to identify the topics I should understand first before working on the task. If possible, I also ask it to provide sources—such as blogs or articles that I can read to understand those topics. I go through those resources and build the necessary understanding before starting the actual task. Then, while working, I continue referring back to it and asking questions such as, “Why is this saying this?” or “Why is this happening?” It tries to explain things based on my existing level of knowledge, so that I can understand the explanation. I also question the code it generates. If something doesn’t make sense to me, I ask whether it is actually supposed to work that way.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: I mostly evaluate the generated code based on its output. First, I check whether it is producing the expected output. Then I go through the code myself and try to understand whether everything has actually been implemented correctly. If I see that everything is implemented properly and that the implementation matches my existing knowledge—or the new knowledge I’ve gained while working on the task—then I keep the code. But sometimes I may question the AI about a particular implementation. For example, I might ask, “Why are you doing this? Shouldn’t it work this other way?” If it responds by saying, “Yes, you are correct. The code I generated was wrong,” then my trust in the generated code decreases a little. At that point, I start questioning how reliable the generated code actually is. As a result, I may stop relying on that generated solution, or I may not use the original generated implementation at all. So that’s where I experience a trust issue—the moment the AI itself acknowledges that its initial implementation was incorrect.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: I run the code against my test cases and check whether the result I actually need is being produced correctly.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: I feel that AI-generated code is relatively easy to review. Firstly, AI-generated code is often well documented. It usually explains what different parts are doing and why they are being done. The comments make it easier to understand. Secondly, when we continuously use an agent or LLM for code generation, over time it starts adapting to our coding practices. It learns the way we structure and write code and starts generating code in a similar style. So, for me, AI-generated code is easier to understand because it is often written in a style that I'm already familiar with. With another human developer's code, however, it can sometimes be more difficult. Their code may not be as well documented, their coding practices may be different, or their coding style may be different from mine. Because of that, I sometimes find other people's code more difficult to understand.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: There is definitely a risk of data breaches or data loss. When we are working with personal or sensitive data, we are often giving the AI access to a large portion of our files and even files in our development environment. The AI might say that it cannot read certain environment files or sensitive information. But as users, we don't necessarily know exactly how much access is actually being maintained or how those boundaries are enforced internally. So there is definitely a data-breach and data-privacy risk associated with this. I feel that this is something people are currently overlooking to some extent.

P11:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I use both Paid and Free. I use ChatGPT Free for general conversations, and I use Claude Code for coding work. Claude Code is given access to the entire codebase.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: A lot of my behavior has changed over the last six months.

Let me first give my answer from six months ago. Six months ago, I used to code manually. Whenever I got stuck somewhere or felt that something was a little difficult, I would go to AI. But now I start with AI. Before, I used to think about how to solve the problem. Now, the first thing that comes to my mind is how I can explain the problem to AI in a better way. I already understand the problem because I have a lot of quality context. So now my starting point is: how can I explain this entire context to AI through an MD file? Thinking about the solution is no longer my starting point.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: This has happened several times. My current project is a monolith, meaning the frontend and backend are all together. So, while working on a feature, I sometimes find that something is missing on the backend. Then AI starts changing Python files even though I am not supposed to touch those files. So the solution I maintain in this situation is this: if I say that my role is frontend, then I should not touch anything on the backend. Instead, AI should tell me what questions I need to ask the backend team. It should put those questions into an MD file for me. This is the approach I follow. Whenever I see that it has changed a Python file that I am not supposed to touch, I tell it, “No, this is outside my scope. Undo that change.” Then I ask it to tell me what I should ask the backend team so that they can add whatever is missing. Basically, it tells me exactly what I should ask the backend team. That is how I handle it. What I have mainly noticed is that this happens because of a lack of context. If I properly tell it that my role is frontend and that it should never touch the backend, and instead should give me the questions I need to ask the backend team, then if I put these instructions in the MD file from the beginning, it almost never goes outside my scope. The tendency to go outside the scope becomes almost zero.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: Sometimes, if AI becomes too ambiguous, it starts hallucinating and giving incorrect things. That is one issue. Another thing I have noticed is with Claude. Sometimes it brings up a pop-up asking for my decision. The pop-up says something like, “This is currently structured this way. Which one do you want? Should I do this? Or this? Or this?” So it gives me three or four options and marks one of them as recommended—“This is recommended.” Usually, I follow its recommended option. But when it does not recommend any particular option, that is when I notice it tends to hallucinate and produce garbage features or garbage code.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: I maintain several layers in this process. Personally, I use Claude Code for coding, but I never give it instructions in my raw language. I first go to ChatGPT. I explain things to ChatGPT in a very rough, almost gibberish-like way: “I need this. I want this.” And if I have a clear idea, I also tell it which files it should check and understand. For example, “Go through this file. Then I need this type of feature—build it.” So I try to mention the file names as well. At the beginning, I give ChatGPT a rough prompt with the file names and tell it, “Make this an efficient prompt for agents.” Then ChatGPT creates a very nice MD file. I also tell ChatGPT, “Give the output in MD file format so that I can copy-paste it directly.” It gives me the output in a proper MD-file format, which I can directly copy-paste. Then I either copy-paste it directly into Claude, or save it as an MD file and tell Claude, “Go through that file and implement the task.” So that is my main workflow. Basically, I first use ChatGPT to understand and structure the task precisely, and then I copy-paste that instruction into Claude.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: This has happened many times—we have forgotten ourselves why we did something. We don't even remember it anymore. In the case of AI, particularly Claude Code, it can reverse-engineer the codebase very precisely and understand what the feature actually was. In around 95–99% of cases, I use Claude Code with the codebase to understand the feature myself first. I would say I do this around 90% of the time. Now I often don't even ask anyone. I just give Claude the file name and folder name and say, “Give me an analysis report of this feature.” This is the flow I personally follow. For example, we had a difficult task in the Contract Module. I knew nothing about that feature. I simply gave Claude Code the folder path of the Contract Module and gave it a prompt: “Analyze this and create multiple report.md files so that I can understand what this feature is.” In this case, Claude first goes through the code, reverse-engineers it, understands the feature, and then explains it to me. So this problem is largely solved. And I have seen many times that Claude Code can explain the feature in a much better way than another developer's understanding—especially through the MD files. Another thing I do is that Claude Code's output—whether it is chat output or MD-file output—can

sometimes feel like difficult English to me. So I give Claude Code's output back to ChatGPT and say, “Make me understand this in simple English” or “Give me a simple explanation.” Then ChatGPT breaks it down and explains it to me step by step. So whether I forget something or someone else forgets something, it doesn't matter that much in my case. Because ultimately, I can use Claude Code to understand the whole thing again. There may be a handful of edge cases—maybe 5%. But Claude can now explain an unknown feature better than most humans. The reason is that when humans explain something, they often forget to mention certain details that were in their head but didn't come out while explaining it to someone else. Claude doesn't really have that problem. It gives a very proper and systematic explanation. There may be around 5% extreme edge cases. For example, there might be a hardcoded value that the Business Analyst specifically requested and said, “This thing has to be exactly like this.” That kind of context may not always be possible to understand from the codebase alone. Those kinds of cases would fall into that 5%. But in around 95% of cases, Claude can explore the codebase extremely well and reverse-engineer an unknown feature.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: This generally puts a lot of responsibility on AI. But what I do now is that I have one main task: I need to understand the domain, my feature, or at least the business logic of whatever work is expected from me. If I can somehow understand or become clear about the business logic, then the actual work is almost done. Because once I understand the business logic, I can clearly explain that business logic to Claude or provide it with the necessary context. Then it can handle the implementation in its own way. Normally, suppose I am given a completely new codebase. I know nothing about it. In that case, I tell Claude to explore and analyze the codebase. On the other hand, suppose the codebase is familiar to me, but I am asked to add a new feature to a module I have never worked on. In that case, I tell Claude, “This is the module, and this is the codebase.” I already understand the codebase and know roughly where the relevant code is. Then the first thing I ask is, “Analyze everything in this folder and create 5–6 MD files for me.”

One could be analysis.md, another could be frontend.md. If Claude has any ambiguity, I even ask it to create a separate MD file explaining those ambiguities. That way, I can ask the BA or backend team specific questions and clearly understand the idea or business logic behind the feature. So that's basically it. Even if the work is outside my domain, I make sure that I understand the business logic of the feature. I still don't have enough confidence to move forward directly with a completely unfamiliar stack. When working with an unfamiliar stack, I will at least get some basic understanding first. I may not understand all the syntax, but I will understand the fundamentals—for example, how FastAPI works, how it communicates with the database, what its best practices are, and so on. If my knowledge is literally zero, I don't think I would immediately start working with AI. I wouldn't get involved in the implementation right

away. Before getting involved, even if AI can help me, I would still study the technology a little. I would gain some basic, surface-level knowledge first and then start implementing. If I have absolutely zero knowledge—for example, if I have never written even one line of FastAPI code and know nothing about it—I would try not to touch it. My first approach would be to skip or avoid it as much as possible. The reason is that I am still not confident about having AI do the work when I have absolutely zero knowledge of the technology.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: The answer depends on 3–4 things. And when those 3–4 things change, the answer changes as well. One factor is: how urgent is it for me? Is it needed right now? Because sometimes we don't understand AI-generated code. Since AI may change many files, we may not understand how everything connects. So one of those factors is: do I need this right now? If it is immediately needed, I make sure the feature is working. At that point, I may not try to understand the entire code. Of course, I need to understand it later, but immediately I may not want to spend time understanding it. That's one case. Another factor is whether I have a time constraint. Suppose I took two days from the client or team to deliver a feature. AI might finish it within 2–3 hours. In that case, I try to understand the code line by line. I manually add console logs. I stage the AI-generated changes and try to go through them manually. For example: “Okay, this happened because of this. This happened because of this. This didn't happen because of this. That's why it created this function. That's why it created these three helper functions.” I try to understand those things. Basically, I try to make sure that if I had implemented it manually, I would have done it in roughly the same way. If I have the time and opportunity, I take it to that level. I use commands to inspect it, or I understand it myself. I try to have a clear picture of the code flow in my head—this happens, then this happens, then this happens. The problem comes when I have absolutely no time, or I'm feeling very lazy, or I don't have the energy. Then I simply check whether the feature works, whether the relevant things are working, and I check some edge cases—especially the cases where I expect it to break. If it doesn't break, the feature works, and it is super urgent and has to be delivered immediately, then I don't review the code deeply. I don't spend much time on refactoring either. I just pass what is there. But the practice should be that when AI writes the code, you manually go through that code. It should be as if you already have the complete idea in your head. Even if AI writes the code, you should have an understanding of the overall codebase and its flow in your head.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Our frontend domain is relatively small. There isn't that much complexity. For example, in the UI, we can simply click around and see what happens—what happens when we do this, what happens when we do that, what happens if we don't select something. The frontend domain is relatively limited. Roughly 50–60% of the edge cases are already predetermined. And

checking things on the frontend is easy because the UI is right in front of you. You can click around and see what happens. That's basically how we check it. There is also something I have heard several people say in relation to the previous question. They say that if you don't understand the code yourself and that code goes into production, then if a bug appears later, you will obviously have to use AI again to solve it because you don't understand the code. In that situation, AI can hallucinate a lot. It may fix the wrong place. It may make a fix somewhere that causes two other things to break. So many people say that if you push code to production without understanding it or properly reviewing it, and—God forbid—a major bug appears, especially during a critical time, there is a serious chance that you will get into trouble. So, to avoid that risk, the right practice is to thoroughly review AI-generated code. And what I'm seeing is that this is becoming a modern bottleneck. When you have a large feature and AI changes 8–10 files, reviewing those changes can take almost as much time as writing the code itself. Previously, code review didn't take that much time. Why? Because we all wrote code manually. We went through the entire journey while writing the code. So we didn't have to go through that journey again during review. But now, that journey doesn't happen while writing the code. You have to go through it during code review. So one thing to keep in mind is that if completely unreviewed code goes into production and a specific bug appears later, fixing that bug can become a major headache for the company or the developer. Because the AI generated the entire code, but the developer doesn't have a complete understanding of that code. That's a major problem. Something I personally felt for the first time is that there is a temptation. You think, “The work is already done, the code has already been written. It's going to work anyway. Let's just ship it.” So far, I haven't developed a large individual product. I've developed individual features. So I haven't really faced that problem at a large scale. Usually, the changes were only across 4–5 files, so reviewing them was easy. But then I saw that you can take a feature that would normally take two months for an entire module and build it in 3–4 days by chatting with AI for 3–4 days. And then you realize that it has changed 70–80 files. At that point, you have to review the entire codebase. My personal experience is that I had AI write 65,000 lines of code in five days. I added 65,000 lines of code in five days. Now, reviewing that code has been going on for two weeks, and I still haven't been able to visit all of the files. Adding 65,000 lines of code is a huge amount. So these are the kinds of problems that come with it.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: Some parts are the same. But I have noticed an industry practice. I still follow the same approach—I manually go through the code, skim through it, and look at the code alongside it. But recently, over the last one or two weeks, I have been exploring this and noticed an industry practice. People are saying that if you generate code using Claude Code, you can then use GLM or Codex to review that code—basically, use another agent. Because one agent can often identify another agent's flaws and edge cases very well. So an industry practice I am noticing is that one agent, such as Claude Code, generates the code, and another agent—Codex,

GLM, DeepSeek, or something similar—reviews the code generated by Claude. And you can do this within your session, directly inside your IDE. The second agent reviews the code and gives you a judgment. This is where experience or seniority becomes important. For example, GLM or Codex may sometimes identify cases that are actually invalid. You know that those cases can never happen in your system. So, as a senior resource, you have to understand which parts of the review are invalid. This is where seniority and experience play a role. The agent may identify many issues, but you have to determine which ones are actually invalid. At the same time, you also have to understand that some of the issues may actually be valid. So you have to make that trade-off. So there is now another practice emerging in code review where the review itself isn't necessarily done entirely manually. I explored this a little and saw people generating code with Claude Code and then having Codex review that code—asking it to review the code and look for edge cases. So this is one practice I have noticed in code review. I haven't fully adopted this approach yet, but I plan to move in that direction. And yes, human review is still required.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: All of my experience is frontend-related. In frontend, I haven't encountered anything that Claude did poorly where I would have done a better job myself. Especially with CSS or animations—yes, it consumes more tokens, so you do spend more tokens. But in frontend, I haven't seen anything that I could say, “It did this badly.” Frontend, specifically React and the React ecosystem, is something it handles extremely well. It does it very well and produces very clean results. And most of the time, I actually learn from it. I learn through reverse engineering. Rather than asking it questions, I often learn by reverse-engineering what it has done. I don't have much knowledge of backend. I also don't have much knowledge of DevOps. So I can't really comment on those areas. But in frontend, I haven't found anything that it does poorly where I could say I would have done it better. This was actually something I experienced around two years ago—at that time, AI used to do some things poorly in frontend. But I don't really see that anymore. There can be company-specific risk factors. For example, suppose a company has its own SaaS product. When I give the code to Codex or Claude, they get access to the entire repository. The product is essentially exposed to the AI. It may potentially be stored somewhere on their servers. Obviously, suppose I work on Microsoft Teams. If Anthropic gets access to my entire codebase and knows everything about the product, as the product owner of Microsoft Teams, I may not want that. So there can be a problem on that end. That's mainly a business-side problem. Many companies may not want their code to go to Anthropic or any other third party, or may not want a third party to go through their codebase. So I do see this kind of privacy-related problem. But from the developer side, in terms of coding itself, I don't really see a problem. If the company doesn't want a third party to go through its codebase, then yes, that's a problem. But as a developer, I don't see a fundamental problem with using AI for coding.

P12:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Basically, I'm currently using the paid version of an AI tool for coding. Claude is my primary coding tool. Occasionally, if I reach my session limit, I use Gemini instead. Overall, these are the two tools I mainly use for coding.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: When I receive a requirement, one advantage I have now, since I use the paid version of Claude, is that I can use its Web Search capability. Previously, in situations where I would use Google Search, I now use Claude's Web Search as an alternative. And if something is more intensive—meaning I need to access multiple websites and do more extensive research—I use the Research option. Through these two options, I first try to understand what the industry standard is—how this type of problem is generally solved in the industry and what technologies or services are commonly used. Whenever I get a new requirement, the first thing I do is often research it using Claude. For example, suppose I need to build an ML time-series forecasting feature. I would use Claude to understand what Amazon or other companies/services use for dashboards, which services are free, which are paid, which would be better for me if I need to scale in the future, which would be more effective, and so on. So, for a new task, the first step in understanding the requirement is largely research through Claude. Even when the task is familiar, I still try to understand which approach would align best with my existing codebase. After that, I usually generate an MD file or implementation plan. Then I give that MD file to Claude Code and have it implement the feature step by step.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: This situation has happened to me personally. For example, in backend development, I work more with backend, whereas my understanding of frontend is comparatively limited. Suppose I need to implement a backend feature. I understand the basic requirement—what the feature needs to do—but I don't personally know what the detailed system design for that feature should look like. If I give Claude a very vague instruction like, "I need this feature," I might end up with Claude writing 3,000–4,000 lines of code. Then, when I go through the code for review, I realize that I don't fully understand what is happening, and there are also many unnecessary things that have been implemented. So I changed my approach. Similar to what I mentioned earlier about decision-making, I first tell Claude: I have this feature, but before implementing it, explain how it works, what the system design should look like, and what the different components are. At this stage, I don't give it my codebase. I only use it to gain the prerequisite knowledge. Once I have an understanding of the system design, I can identify what specifically needs to be implemented. For example, I might then realize that rate limiting is required. Or I might identify that a particular database query is unnecessary, that multiple database queries are happening, or that a database connection is being kept alive throughout the process and causing unnecessary delays, so some optimization is required. So, I first gain the

prerequisite knowledge and then make my prompt much more specific. After doing that, I found that the feature implementation was much more properly aligned with what I actually wanted. What used to be very verbose became much more precise—only implementing exactly what was necessary. I've realized that if I describe a feature requirement very vaguely, Claude can sometimes generate thousands of lines of code. But the more precise I become—for example, saying that a load balancer is required here, or that a specific database query is needed for this particular operation—the more accurately Claude can implement it. So, the question itself needs to become specific.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: What I have noticed is that if I don't explicitly tell Claude to clarify things—for example, "For each state, clarify what you are going to implement"—it usually doesn't ask me for clarification. I normally start my prompt by saying, "Implement this." Then it will usually start implementing immediately. So now, whenever I give it a prompt, I try to explicitly say: "If there is any blocker or confusion, ask clarifying questions." Because if I don't explicitly mention this, it generally starts making assumptions and proceeds with the implementation. Then I have to go back later and fix the same things repeatedly. Another thing I try to do is maintain a separate test file for edge cases. For a particular feature, I create a separate test file. Once the feature is implemented, I run those test files repeatedly and check whether all the tests are passing and behaving correctly. Afterwards, I delete those test files. They are basically for my own convenience during development. I don't use them in production. So, there are two things: 1. 1. Pushing the AI to ask clarifying questions. 2.Writing separate test files to test edge cases. These two things sometimes help me handle edge cases better.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: First, when I give a prompt, I try to provide as much detail as possible from my own side and not completely depend on AI for that. Suppose a feature requirement comes to me as just one sentence. I first think about how I can make that requirement as detailed as possible myself. That could be the system design, feature details, or any other requirement-related information. Then I use AI—for example, there is an extension called Superpowers, and it has an additional superpower for brainstorming. I use that to brainstorm and structure the plan first. I explain in detail: this is my system design requirement, this is my knowledge requirement, this is my business requirement—based on these, generate a plan. Then the AI brainstorms and creates a plan. I read that plan very carefully and check every line—whether any requirement has changed, whether anything is missing, or whether there are any incorrect assumptions. Then I comment on the plan and edit it again. At the same time, I ask the AI for its opinion—for example, "Am I missing anything?" or "Do you see any scope for improvement?" The AI then provides additional suggestions. Through this repeated process of commenting, reviewing, and refining the plan, I eventually finalize it. Only after the plan is finalized do I assign the actual implementation work.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: This has happened to me many times personally. A lot of the time, requirements don't come directly from the developer side; they come from non-technical managers. So the requirements are often quite vague. We implement them, but afterwards we sometimes can't even determine whether the requirement was actually fulfilled properly or whether we implemented the right thing. Then reviewing the code again and again takes a lot of time. And Claude's tendency to implement unnecessary things is still there as well. So when the requirement is vague, I frequently face situations where unnecessary things get included during implementation. This is something I face fairly regularly, and later I have to go back and remove those redundant parts. Another thing I've noticed is that Claude sometimes doesn't make the code properly modular. So I have to explicitly force it to make things modular—for example, by creating a reusable function so that the same logic can be called from multiple places. I have to tell it not to implement the same function or logic separately in every file. I've encountered situations like this as well. Basically, when the requirement or plan isn't properly finalized, I face these kinds of problems much more frequently. A very recent example is something we are currently planning: we're trying to make the way the Dev Team handles deal creation in [company] more agentic. The requirement can basically be summarized in one sentence: deal creation needs to be made agentic. Until now, people have been manually creating deals, and the goal is to make that process agentic. That's essentially the entire requirement we've been given. But the problem is that my own understanding of the requirement isn't fully finalized yet. In some places, I'm realizing that the manual approach might actually be better. In other cases, I'm realizing that there is no real reason to introduce an agentic approach at all. So when I try to implement things repeatedly without clarifying or finalizing these decisions, I run into the same problems again—which parts should actually be agentic, which parts should remain manual, and where there is simply no reason to introduce an agentic approach. This is something I've been struggling with recently. For example, suppose there is a discount model. There isn't much detailed documentation about the underlying equations, calculations, or the overall logic of the discount model. There is also very little documentation around forecasting. It seems that everyone simply knows that the system works in a certain way, so they implement it that way. But we don't actually know what calculations are happening behind the scenes. Because of this, when I was trying to build something new for forecasting, I had to make a lot of assumptions. I assumed that the calculation probably worked in a certain way, so I instructed the AI to implement it accordingly. The AI then generated the pricing or forecasting code based on those assumptions. But I kept seeing that the dates or figures that should have been produced using the same calculation weren't actually matching. Then I had to go back to the original codebase and figure out how the existing implementation actually works. Sometimes I don't have the full context or access either, which makes it even harder to explain the calculation to the AI. Eventually, I had to manually write down the calculations in a working file or an .md file. Then I gave that to the AI and asked it to verify the implementation—for example, "The calculation actually works like this, so make sure your implementation follows this exact logic as well." This is the area I've been struggling with quite a lot recently.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: In general, I try to follow the approach people used for implementation before AI became common. This has helped me quite a lot, especially in the case of [company]. Suppose I recently started working on deal creation. To understand how the whole thing works, I first watch their documentation videos, especially videos from around 2022 or 2023, when they were developing without AI. I try to understand how they used to think, how they understood requirements, and how they created documentation. One advantage of this is that AI models fundamentally produce probabilistic outputs. So, if I first understand the factual documentation or factual information myself, the requirement becomes much clearer to me. Then, if I tell the AI, “Proceed based on what you have understood so far,” it will probabilistically continue in a certain direction. But as the scope or complexity of the task increases, there can be a point where it gradually starts to deviate from the original requirement. That is why, as much as possible, I first gain the knowledge manually and create the documentation myself. Then I provide that manual documentation and the decisions I made to the AI.

For example, in the case of the discount model, I watched all the relevant videos and created the complete documentation myself in Microsoft Word, Google Docs, etc. documented how the system works, how the calculations are performed, and where each equation is being used. Then I gave that documentation to the AI and told it, “This is your strict requirement. You should not divert from this.” This way, I create a factual and manually verified foundation beforehand so that the AI does not gradually move away from the original requirement as it proceeds probabilistically. It does take me a little more time sometimes, but overall, I find this approach helpful.

Interviewer:Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: The first thing I try to achieve is “good enough.” I don't try to over-optimize things unnecessarily, especially when the optimization makes the code harder for me to understand.

At the end of the day, everything has to scale at some point. So I first try to avoid overly optimized implementations that could become difficult to scale or maintain later, whether by AI or by myself. For example, if the AI writes a function, I first check whether the function itself is properly implemented. Then I look at things like modularity. If the function is doing multiple things, I check whether those responsibilities should be separated into multiple functions.
I also check for redundancy. For example, sometimes I see two functions where one takes three parameters and another takes five parameters, even though they are essentially doing the same thing. In that case, I would merge the two functions and adjust the parameters and calls accordingly. When these things are relatively simple and understandable to both me and my teammates, I consider the code more or less ready for production. I assume that AI will increase the size of the codebase to some extent. Trying to make the codebase extremely precise and optimized can become very time-consuming. So instead of spending excessive time on that, I prefer to maintain good functionality documentation. That way, even if someone doesn't

understand the code, they can read the functionality documentation and understand what the feature does, what is supposed to happen, and what should not happen.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: I see this differently depending on the situation. For example, sometimes AI tries to fetch information from documentation, but it doesn't always rely on documentation. Sometimes it may fetch information from an online forum or another source. So, depending on how important the information is, I verify the documentation myself.

For example, I was recently working with an API key. The AI told me that the API allowed around 1,000 calls per day. That seemed unlikely to me, so I went to the official documentation and checked it myself. The documentation had information about rate limiting, which the AI had not mentioned. So, for this kind of situation, I think prerequisite knowledge is very important when verifying an AI's answer. I need to know enough about the subject to recognize whether something sounds reasonable or not. For example, I have been using these tools since university. When I studied concepts like transformers and attention and understood how they work, my prompting also became better. Similarly, as a backend developer, when I first learned how APIs work, how CRUD operations work, and how these systems generally function, it became much easier for me to verify AI-generated information. So I think verifying things without prerequisite knowledge is quite difficult. Usually, AI provides links or sources, so I try to verify whether the source is reliable and whether the information actually comes from the original documentation.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: Most of the time, I feel that having a teammate review something is more time-consuming. That is true. But as humans, we naturally consider many constraints. AI always needs context, whereas humans use many constraints as context almost automatically because they already exist in our understanding. If I want AI to perform a review, I have to provide all of that context beforehand, including the business logic and other relevant requirements. For example, AI can test whether the features mentioned in the PR are working. But there are aspects of manual testing that a human can understand more naturally. As a user or as a person, I can think about the feature from a product manager's perspective and ask whether the implementation actually makes sense.

Because of that, so far I feel that having a teammate review the PR is generally better, even though it is more time-consuming. For example, I was recently working on something involving a lot of database queries. The next step was taking around 10–12 seconds. I had implemented it using AI, and the data itself was correct. But I wondered why it was taking 10–12 seconds. When I manually reviewed it, I realized that many of those query calls were unnecessary. There was an extra query being called, which was increasing the response time. After removing those unnecessary calls and refactoring the implementation, the response time came down to around 1–2 seconds. I could potentially give AI a rule such as “anything taking more than two seconds is bad,” but that doesn't necessarily work. A query might legitimately take more than two seconds

because it is fetching data, performing calculations, or retrieving information from another connector. So if AI simply assumes that anything taking more than two seconds is bad, that would not always be correct. I feel that humans can often identify these things without needing all of that additional context. That is why, for PR reviews, I currently find manual review somewhat better. AI can still be used as a helpful tool to understand and identify issues.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: Regarding risk, as I mentioned earlier, if you don't explicitly specify certain requirements, AI can sometimes hallucinate and leave security vulnerabilities in the implementation.mFor example, I once had to work on a frontend task, even though I have very limited frontend knowledge. I implemented the feature, and after everything appeared to be working correctly, I tested it.mThen I opened the browser's Inspect tools and noticed that all of the client's data was being exposed on the client side. I had not explicitly mentioned security in my prompt. I simply said, “I need this implemented in Next.js. Please implement it.” That was the requirement. Everything appeared to work perfectly from a functional perspective. But the AI had overlooked the security aspect and exposed the client's data. I think this is something that many people can easily overlook, especially security-related issues. We can see similar kinds of problems in different systems, including situations where sensitive information such as card details may become exposed. So, when people blindly focus on feature implementation, security can sometimes be overlooked. I think this is a significant risk. This can happen regardless of whether someone has strong domain knowledge or not. In the case of the client data issue, for example, it was something that most developers would consider basic knowledge or common sense. But the AI wasn't considering that common-sense constraint on its own. That is why I sometimes feel that human PR review is important. A developer reviewing the PR might naturally think about these things. At the same time, developers may also assume that because AI is capable of doing many sophisticated things, it must have already handled basic security or redundancy issues. Because of that assumption, I think developers can sometimes overlook security risks or unnecessary/redundant implementations in AI-generated code.

P13:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I mainly use Claude — basically, I use the paid version of Claude for coding purposes. And for normal documentation writing or any writing needs, I use the free ChatGPT.

Interviewer: Okay, which one — which paid plan?

Participant: For Claude, it's the enterprise plan. It was provided to me by my company, and I use that.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: What we mainly do is, when a codebase comes in front of me, I don't need to change the whole thing. Suddenly a requirement comes — say a database has changed: earlier it was Postgres, now it's an analytical database. For that portion, what I do is I give Claude what was the schema before in Postgres and my new schema. And I tell it which places need to be changed — I mention the function at this place, and tell it to change here and here, and to mention the changes. It does exactly those.

Interviewer: What influences the decision — meaning, generally, based on which considerations do you decide what should now be left to it?

Participant: Mainly, I give some description of the logical part — that this is this, this, this. Then the actual code-writing portion, it does entirely. Before finally pushing or committing, I do a check.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Mainly, I've faced a few situations where I want one thing and it actually over-estimates. For example, with a query — a simple query that could be written in a simple way — it optimizes a lot and adds lots of joins and all sorts of stuff. It ends up overkilling it. I've faced situations like that.

Interviewer: And what do you do then?

Participant: Then, basically, what I do is prompt it once more — 'not this, this; do it this way' — I give another guideline: 'you can go this way.' In some cases that solves it, and in some cases I see that my intervention is needed — I need to tell it more precisely, 'no, do it this way, this way.' Mainly like that, I try to solve it.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: I'll give two examples — one with Claude and one with ChatGPT. With ChatGPT, what happens is it makes an assumption of its own and then gives an output that way. Sometimes

it doesn't cover my scope — in most cases it doesn't cover it. With Claude, what happens is, before taking any decision, it asks — 'I want to do this' — and gives me options: 'in this case, which one do you actually want? One, two, three.' I select any one, and I have to tell it, 'okay, then we'll go in this direction.' For example, let me give a small recent example. I had a task to convert from Postgres to an analytical database. Now, it doesn't have access to the database. So before writing a query, Claude asked me: 'What is your current situation? Does your schema have these columns?' Then I told it, and it wrote it accordingly for me. But ChatGPT didn't recommend that for me. . It didn't make several assumptions and solve it its own way.

Interviewer:When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: Mainly, when I use an agent, I actually tell it the whole thing explicitly at once — 'this is the task, these are the requirements, you can go this way.' And I also take feedback from it: whether there's actually any better option, whether there's a better way. If yes, it analyzes and says, 'yes, there's a better way.' Then I take the solution from it, and consider how much I can optimize if I go this way — and proceed like that.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Basically, what has happened is that I've had to do my work using another developer's code. If it's my own codebase, then in most cases I review the code — when I get code, I review it, and then I commit. But after getting someone else's code — someone else's code that's also AI-generated — then why he did it that way, whether I know about it, whether I can understand it. In this case, if I don't understand — suppose what a function is doing or why it's doing it — then I ask the AI again, to explain the very basics: what this function is actually doing here, or why it does so, whether there's something better.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: The first thing I do is tell it to explain it — what it is, how it works. And I ask several tools. If it's something more vital, then I ask both ChatGPT, DeepSeek and Claude. It gives a solution; then I read that solution and understand it. Then I ask another AI (ChatGPT) is this better than the other option, is there a better solution — like that.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: When it comes to the decision, I actually review, and after reviewing, if it falls within my sense of things, then I can tell that it's okay.

Interviewer: Do you also have another agent check it — you had one agent write it, and do you have another agent check it, like that? Does it happen like that, where you change the context — say you did the implementation of a feature in one context, and then you didn't ask it to check in that same context, but in a separate context you told it to check what work has been done on this branch?

Participant: Yes, this has happened. I work on one branch, then I shift to another branch and tell it that I'll compare — how the output of that branch compares with this one, and what the output of the code I wrote on this branch might be.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Yes, I do verify. When I get a solution, I definitely test it manually to see whether my expected output is coming. And what I mainly do is, after getting a piece of code, I check whether there's an optimized version — at that time I prompt it, asking whether there's an optimized version.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: In most cases, I do. Especially when I give it something — whether it got optimized, and how much — I check that more and more.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: The first problem I feel is — suppose I'm using a tool, and if it's not available, then what happens? I mean, my level of human thinking has dropped a lot. Second, the dependency has become much greater. After that, sometimes if I feel lazy, it turns out I don't review the code properly. I push it directly.

P14:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: For now, I'm using two tools. Both are paid — Claude and Codex. Both are agentic tools that I'm using.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: In terms of what influences it, there are a lot of tasks that are repetitive. For example, when I work on the backend, there's schema definition, and some more things — like setting up every route, or setting up certain things that are repetitive work. In these cases, I fully hand the repetitive work over to the AI. But if it's business logic handling and everything, I do that myself. And if it's repetitive — for example, exception handling: I've defined it once, so afterward I don't need to personally write every exception handler. I give that to Claude or some agent.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Yes, it did. I'm recently working on something for myself. At one point it was given a plan saying we would do subscription-based work here. There, it built various subscription layers on its own, and did the whole implementation on its own, without asking what features there would be, or what kind of things to keep, or what to keep in the free tier. Later, after it built it, yes, a lot of things were good, but a lot of feedback was given, since my motivation was that it should be for everyone. In some cases they get over-excited — they end up doing the planning by themselves.

Interviewer: And then, do you fix it again by giving commands, or do you fix it manually yourself?

Participant: Usually, since it had implemented the whole feature, my feedback was about which tiers could be kept — there was no free tier, so I added that. Like that, basically, the feedback gets fixed by giving it commands.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: First, I check whether it first explains to me the requirement — what I said. Then I tell it, if there's any feedback — like, maybe it didn't understand what I said or what I meant to convey, or it has overthought things — that feedback gets fixed. It does its implementation, and after the implementation, I basically do unit testing on both backend and frontend, for every part. So every unit has testing. Everything is tested according to the requirement.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: This happened. What I mentioned earlier — I had given a command that I want to bring a subscription feature into my app. But that was more like 'let's explore how we can do it.' So Claude basically planned the whole thing on its own and executed it without asking. So later it was hard to understand where this came from. But yes, since I had the idea, and there was one advantage — there was a kind of tracking: it had made a proposal in OpenSpec and worked there — so it was easy to track. But finding…

Interviewer: I mean, the thing is like this: say a previous developer of yours wrote something, and now you have to edit or change it, but you don't know why he did it, why he wrote this function. So at that time, how do you handle it?

Participant: In the current case, this happens frequently — in the project we're working on now. For example, if I tell you today's situation: today a feedback came that this function already exists and I should use it. It's linked up across many files. I could understand by just looking at the function, but where it has been used, whether I can change it, what its dependencies are — understanding that across the whole codebase was difficult for me. In that case, I take help from the tools: I ask it whether there's any dependency with it, why it was implemented, why it's done this way, and whether changing it somewhere else would break something.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: My choice initially was: if some concept isn't clear and I don't have much knowledge of it, I'd hand it over to the AI and have it done. But later, what I do now is this: if I have to implement some feature and I have no idea, I first discuss how it could be done. Then I ask it what keywords would be needed to implement this. Say it tells me that to implement this I'll need Kubernetes. Then, if I have no idea, I ask it: what is Kubernetes, why is it used, why would you use this, what are the alternatives — and from that I understand which is best. Another thing I do here is use a cross-agent tool. Say it says okay, use Kubernetes; I verify that on another platform — whether there's an alternative that's better. And another thing I use is Context7, which helps in figuring out what's appropriate — whether what I actually want is implementable. So first I get it explained, and then I start the implementation.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: For now, I maintain some structure — the structure that defines backend and frontend. After it suggests that, first I check whether it has done the layering properly. Then I check the code to see whether it has been done according to the requirement. Then, whether it has fixed whatever I had in mind or whatever we discussed. The final check is the intention, or everything — or what we call the code standard — whether there's anything like that to check. If there's nothing, then it's approved; otherwise, it's asked to fix that.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: First, in the 90% case, I try to verify whether what I asked for is correct. What I want is — I know the coding; if I say it should be an API, I know what coding needs to be done for an API, so I check that, and if I see that yes, it's there... For example, we commonly know

that dividing by zero will cause an exception. If the basic things like that are in place and it's good to go, then it's accepted, and then various kinds of tests are run.

Interviewer: This testing — is it always manual, or is it done by a second agent?

Participant: First it's manual — manual in the sense that, to give an example again, I give some test cases. But once the test cases are done, if for some reason something has to be changed, then they do regression testing automatically and check those tests. First, they test the test cases once on their own. One thing here is that I also don't consider all the tests myself — many times the agent doesn’t either.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Interviewer (clarifying the question): The thing is, when you become comfortable with an AI, after a certain period, do you stop checking its code? Or do you do the same with a colleague — that always good code comes from them, so okay, you stop reviewing?

Participant: In my case, when it comes to the review section, I look at every piece of code at least once — whether it's from an agent or a colleague. Because the requirement differs — what I wanted versus what happened. So basically it's a sanity check: okay, this happened here, and it needed to happen this way. I have a kind of map in my head — this needs to happen in this place — and I check whether that has happened. Although not checking the full coding, just checking the things that are expected.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: In my opinion, the first thing is doing the entire implementation fully with the agent, without checking. Because, from my personal work experience, many times — one thing is linked up in many places, but when it's changed in only one place, whether it has been changed in the other places too — if this context isn't fully available, and it's a big codebase, it is very easy to break something somewhere. So what I feel is that giving full control of the code is the biggest downside for a developer. Yes — for feature implementation planning, we can depend on an agent, but not for implementation. For planning it's good, but for implementation, keeping control is important.

P15:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: I use the paid version — a Claude Code subscription. When its tokens run out, I sometimes switch to ChatGPT and use whatever free-tier version it allows.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: I actually try to have it attempt almost everything first now, right from the start, because that saves the most time. Mostly I do the debugging myself — figuring out where the code actually is, where the real issue lies. If I see that the implementation is confused, or my instructions were wrong, or for some reason it's genuinely not giving me the expected outcome, then I get more personally involved. In edge cases, I always try that first — if it still can't do it, I try giving it better instructions, which takes more of my own time.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Sometimes the issue is that I don't clearly define the full scope — say I give a task like "change this thing here, make it match this column in this table" — but since it has access to my whole codebase, it sometimes detects other issues beforehand and, if I haven't explicitly told it not to, goes ahead and fixes those too. In some cases that extra fix is actually welcome; in other cases, something is intentionally left as-is for business reasons — the AI doesn't recognize that as "standard" and changes it anyway, and I only catch it during manual testing.
What I did about it was set clear parameters — telling it to always stay within scope — and once I did that, it stopped doing that. But when I start a completely fresh context, it often does this again: fixing something unrelated that it noticed, without being asked. But it writes down that :I have also made this change.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: In unclear cases, Claude Code gives me four options — it builds something like a task list and asks "is this what you want, let me confirm I understand your context." When there are multiple scenarios it's considering, it'll say things like "you could do this statically for now" (and rate that as most recommended), a middle option as less recommended, and a third option it explicitly won't recommend — telling me not to do that. It also gives an option to describe things more precisely myself, to pin down exactly what I mean.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: It does a kind of review step — saying "based on this, I understand you want X" — and if it's still unsure, it asks clarifying/counter-questions. That process takes real time, with it saying things like "please clarify this a bit more, I still don't fully understand."

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: It happens with developers who don't use AI. That person also doesn't always see the full context — for example, in a legacy system with a lot of unfamiliar logic, when AI scans the code it can pick up that logic in one pass; another developer also using Claude Code on a similar scan gets similar clarity. But a developer who doesn't use AI sometimes misses a scenerio— like a function built a certain way in another file, for reasons that aren't obvious from just reading it, conflicting with a current change — so they end up fixing it manually without ever fully knowing why it was originally written that way. Sometimes that's just an unavoidable drawback of how the current system is structured. That time the developer becomes confused why it was done this way. So if the explanation is given, then the developer can resolve it.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: If I'm generating an idea completely from scratch, I use "plan mode" — I go into plan mode and discuss my idea with it, telling it to leverage a certain approach, and it responds with what could be done, including suggesting a database if relevant. For a better-grounded suggestion, it can produce something like a plan-note.md file, which I then review — if it matches my understanding, I tell it to go ahead and generate the implementation. If I still don't fully understand, I rely on more back-and-forth discussion, especially if it explains things in more human, plain language.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: If I'm pulling from cached/previous context, I sometimes take a standard coding instruction from there, and I read through the instructions. The way I used to code before — we all have a folder structure, a coding standard — I like to keep to a concept: try to make things as reusable as possible. So we keep something like a code-instructions.md file with all our conventions: whatever you build shouldn't go outside those conventions. Double-checking is fine, but working outside that policy is not.
If the instructions are clear, it generally goes well roughly 90% of the time — I break tasks down into smaller pieces. Some people, since they're coding with it anyway, decide they don't need to split things up that much and just do it all in one prompt — so if the AI's instructions aren't clear, it may also just do everything in one place. Overall, how well this goes depends a lot on how clear the given instructions are.

Interviewer: As you've become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: In my case, for really basic, repetitive features — like a simple submit button on a form, nothing complex — I usually don't request a review from anyone else either, because the work moves through very quickly; having a human review something that routine would be a very time-consuming process for little benefit. If a small issue does slip through, it's not worth reviewing for — it usually gets caught during testing instead, and we resolve it off a checklist afterward.
But if something is genuinely complex or touches important business logic, I do sometimes share it with a peer — asking them to just check whether a particular part is fine, or whether a change might break something elsewhere.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you've personally experienced that you think developers may overlook when using AI?
Participant: The problem is that things constantly change over time — for example, on day one I planned with Claude and took a subscription, and ran into a real crisis with tokens: even a small task would burn through them completely, and wait times varied a lot (five hours in one case; forty minutes of use and my tokens were gone, then a four-hour wait). Looking back, some work I did in one long stretch could have been broken into smaller two-hour chunks, using fewer tokens per session and getting more done overall. Early on, using a strong model for about 30 minutes could finish a task, so now what I do is have the planning phase done by the best available model — because its judgment is the most reliable — so that less token is burnt during implementation. For small bugs, smaller/cheaper models like Haiku can often handle a lot of it too.
From a human standpoint, I sometimes get confused about what to ask for next — I don't always know yet what the AI is capable of building or what still needs to be built.

P16:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: Normally, I'm not currently using any paid tool. But I do have some experience. Can I mention the name of the tool? There was an AI called Trae AI. It works in agentic mode. I used that for a month for demo purposes. So currently I use conversational tools — ChatGPT, Gemini –this type of tools.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: Normally, since I don't use agentic tools right now, I still don't give it full control. What I do is, when I work on a project, starting from the basic boilerplate, the basic business domain of my project, I develop that myself up to a certain level. After developing it — like in one project I was doing where I used AI — after developing up to a certain level, I gave it control, saying okay, I need changes in this place — that kind of control.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: When I define the scope of something — 'you do this much for me, just this much' — in that case it doesn't generate anything extra. But suppose I said something like 'build me a multi-select input field.' In that case, I might have said it that way, but I haven't properly described what features it should have. So it might happen that after building it, it also added keyboard bindings, which I didn't want, or which I could have done myself — this type of thing it does extra.

Interviewer: Then what do you do?

Participant: Then I check whether the thing actually fits with my design, or my business domain, or my current expectation — whether the new thing it added is something I want.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: This is something the current agentic ones do — I've faced this. When I ask even Claude's browser version, or Lovable, when I tell them something like 'build me something,' they ask us some questions properly. When something hasn't been properly specified to it, it asks questions, and if I agree, then tries to build based on that.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: Okay, in this case, I normally try to give as small a prompt as possible. So that's the first thing from my side. And even if I've given a big prompt, it's generating something for me — generating code — so by looking at that, I can actually tell whether it has understood what I wanted.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: I faced an interesting incident like this. One day my team lead asked me to do a task — to fix something in one place. This was basically for demo purposes. So I opened its codebase and saw that the whole code was more or less AI-generated. A coworker of mine had been doing the work at that time. After he did it, I went and saw that everything they had built, including tables and whatever else, was done using only HTML div tags — only div tags, I mean no other proper HTML tags were used. Then I couldn't understand the thing, and when I went to fix it, I saw that actually nothing was proper there. It was an experience where I later ended up changing the whole thing properly, almost completely.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you?

Participant: In this case, I divide the process into two steps. In the first step, using a conversational tool, I try to understand the business domain — what I actually have to do. I build my own background, and if there's any written document in the project I'm working on, I try to go through that. Once I have a reasonable understanding, I then check with other AI to move with the process.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: As I've already said, normally I don't give big prompts — I don't generate a huge feature all at once. I generate small parts. Since my prompts are small, or are for specific tasks, the AI-generated code is also very small. So it's not much of a hassle for me to review the code. So I manually review the code, and after that I use it.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Along with that, since I generate small prompts, I can understand the generated code. If it ever suggests something, in many cases that's also easier for me to identify — okay, the extra things it's doing, yeah, this could be like that. Then I review it, and I also manually test it myself — I mean, dev test.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed? Is the process different from reviewing a colleague's work?

Participant: I understand. Yes — for me it's now similar, because my usage is, like I said, in small portions. Since I don't do AI-driven or fully AI-based code, to me it's quite similar to how a coworker or teammate would review it — we review in that same way. If I have small pieces of code, it's a bit easier; normally I check it myself, and then proceed.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: Normally what happens is, I give it something and it generates code for me. In the current situation, it doesn't generate wrong code — that's not the risk. What I feel is that, with the scope of my business, it often can't connect to that scope. Suppose it builds a standard thing, but for my current implementation, my current situation, that standard thing is often not valid for my current project, or a somewhat different implementation is needed. So I feel this is the risk — if I hand over full control to it, it might miss this. To me, so far, this seems the big concern, because everything for us is actually scoped on a project basis.

P17:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions?

Participant: I am currently using Claude. These days I use it for my codebase most of the time. Design, planning, structure, db design - these types of tasks are done by me. For conversational discussion with AI - Claude, Codex, ChatGPT is used by me. Out of these scopes, for general purposes sometimes I use Gemini. Codebase related tasks are done by Claude.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: When I can clearly define the solution myself, I tell it what the problem statement is, how to solve it, and how to test it. It simply follows those instructions and completes the task accordingly.

As for thinking help — for example, during planning, I have to explain my general-purpose requirements to it, and it thinks things through based on that. Then that gets turned into documentation. From the documentation — how things will be designed, what the backend structure will be, how data will be handled — it tells me what the best approach would be at that point, or which tool would work better. In most cases, I select the tools myself. Still, say we need to use an ORM for the database — it will suggest that in this language, this ORM is better, that it's a better option than another one, since it can read documentation very easily. Decisions of this type are taken from it. But the big decisions, the major ones, like how the project requirements will be — the final call — those come from me. It writes most of the codebase, but testing and the part where the code is read line by line are definitely reviewed by me.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Yes, I've encountered this scenario. Sometimes, I tell the AI that there's an issue in a particular file and ask it to investigate. In some cases, it ends up fixing other similar issues in the project as well, even though I didn't ask it to. That's actually a problem. This tends to happen when I give it full permissions. So now, when writing prompts, I explicitly include instructions like, "Don't change anything else." Being specific about the scope helps it perform better.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: The AI definitely asks clarifying questions. It presents different possible approaches and asks me to choose which one I prefer. These often come in the form of

multiple-choice questionnaires. After getting answers from there, it starts working based on my responses.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: That's why having clear business requirements is essential, regardless of whether we're working on the frontend or backend. Each has its own requirements, and the technical requirements are then derived from the business requirements. After that, the architecture is defined. This gives the AI a clear understanding of the project context, the tasks involved, and how different changes might affect the rest of the system. Once it has that level of understanding, things generally go much more smoothly.

Interviewer:Can you remember a situation where you or another developer later had to change AI-generated code but did not know the original reason behind the implementation? What happened?

Participant: No, I haven't faced this type of problem. For example, if I ask the AI to explain the purpose of each file, it does a great job. It can also provide a detailed, in-depth explanation of the overall project, so I don't face any issues with understanding the codebase, even when certain parts of the work haven't been explicitly explained to me.

Interviewer: When you have to work on something you do not know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: If the concept or requirements aren't clear, I don't use AI for that task at all, because working on it without clarity would just waste tokens.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: The main decision here is how strictly I want to enforce the project's architecture. I can either keep it strict or allow some flexibility in the usual workflow.

For example, if the project follows a layered architecture, I can enforce strict separation between layers, such as the controller, business, and data access layers. In strict mode, the AI must follow these boundaries and cannot bypass the business layer to access the data layer directly from the controller.

However, if the project has a tight deadline, I might allow more flexibility and tell the AI to work in flexible mode. In strict mode, I explicitly define which layer each file belongs to and require the code to follow that structure without deviation.

Ultimately, the choice between strict and flexible mode depends on the project's requirements and timeline.

Interviewer: When you accept a suggestion or solution from ChatGPT or an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Yes. The first thing I do is review the changes. When I used to write code manually, if I wrote ten files, I knew what was written in each of those files. Now, if the AI changes five files, I go through those changes in the codebase to see exactly what it has added or modified. If I come across anything unfamiliar, I explore it a bit and try to understand how it works. Once I understand the changes, I run the codebase and test it.

Interviewer: As you have become more familiar or comfortable with AI tools, has the way you verify their output changed?

Participant: No. As for the AI's testing — it writes all the test cases itself, whether for the frontend layer or the backend layer; it does it from both layers. But the testing I do on my end, I obviously still do — I never let that slip.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you have personally experienced that you think developers may overlook when using AI?

Participant: There's really no risk in this. Since we have them write the codebase while maintaining their privacy policy, and they themselves say they won't share it. At a certain level, when they ask whether they can use the project for their own research purposes, there's a permission layer. We share based on our own choice. Ultimately, we've seen that the work gets done faster than before. That's a huge benefit. I don't see any downside.

P18:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: First, for an agent I use Claude — I've had the Max subscription from the start. The other one I use is ChatGPT, the $20 version Grok. I don't use its Codex; I use it a lot for the chat version, basically when I'm working at the architecture level.
I use Claude as my vibe coding tool, and I always give it full access within a project. But to reduce token usage, for any bug or fix I usually point it to exactly where the work is: in this file of the code, this was done; in this file, that was done. When I tell it that, it can work faster and uses far fewer tokens. When I just describe it vaguely, I've seen that it takes longer and uses more tokens scanning the whole codebase.
And for chat, I use ChatGPT and Claude too — mostly to compare them and see which one gives the better answer.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: What I actually do these days is, for any task, I start the whole conversation with the coding agent, from decision-making and planning onward. I start by discussing it with the agent, then review it myself to check whether it matches what I want and whether it's right. Then I ask it to write the plan; for that plan I basically use OpenSpec. From there, I hand the whole task over to it.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: Yes, it has done a lot of things like that. For example, standard practices: it follows them even when I don't ask. In one project I was going to use OG / OG links; the standard size for that is 1200 × 630, which Facebook uses. I never mentioned this, but it assumed it and worked to that on its own. Later, when I went in to check, I saw it was following the standard. It maintains standards like this for a lot of things.
As for the negative side: if I don't specifically tell it the standard or other details, it works a bit roughly. Especially when it comes to code structure, and second, optimization. For example, a single API would do the job, but it ends up building multiple APIs.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: This whole thing depends on the settings: whether I want to work in auto mode or in asking mode. It's one or the other. So I always do the complex things in asking mode, such as architecture-level decisions and planning. When it moves on to coding, I mostly use auto mode, and it does whatever it needs to. That's because I have my guidance skills set up, covering what

kind of structure it should follow and what kind of design patterns it should use. All of that is already in place.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: For this, I usually follow one approach, I give it an example scenario whose solution I already know. Then I see how it works through that scenario's flow according to its plan, asking, "How will this flow work in this plan?" If I see it drifting in that flow or adding unnecessary things, I guide it: this isn't needed, this phase isn't needed, do it without this. That's how I verify. I mainly take an example-based approach: I give it an example and check how it solves it.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: That has happened a lot at the junior level. Recently, I ran into problems while contributing to a project. For example, it was a modular monolith, and we had defined the architecture for each module: the complex ones used clean architecture, and the ones that weren't complex used a three-layer architecture. Even with the screen in place, they had built some APIs in a hardcoded way, meaning hardcoded using AI, as a layered architecture. I had to deal with issues like these.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: No, if I don't know it, I never do that. If I do know it, then I just hand it over; it's a simple thing.
For example, we got a requirement for HR document management. It involved electronic document management: large PDF documents that need to be searchable, with OCR, and so on. I was unsure which tool would be the right fit. If I had left it open, it would have just started with any tool, I know that. So I clarified that with it first. It gave two suggestions: Paperless-ngx and Mayan EDMS. Before choosing, I learned what Paperless-ngx does and how it works, and after checking that it matched my scenario, I selected it.
For common things: I was working on a small app for guest management for invitations. Nothing very complex. For that, I generated a PRD, generated the requirements, and left it completely to the agent. There was nothing complex there, and I knew which tools would or wouldn't be used.

Interviewer: So when nothing is complex, you leave it to the agent, and when something involves a decision, you yourself…

Participant: I keep control of it. I do the R&D on it myself first, build a small project to try it out. Then I sit down with the agent again.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: I do trust the model. What matters to me in these cases is whether something could cause trouble in the future. Say, for example, it has written a complex API, and my experience tells me that using a design pattern would make it more scalable and make future additions easier. These days I don't even need to go into the code. I just ask it: "How did you do this portion? Give me that part of the codebase with an explanation." It gives me that, and I see that it has done it roughly, with no design pattern. Then I tell it to apply one.
In the review process, I also do one more thing, I make small reviews and, if there's no time pressure, commit in small pieces. That doesn't happen very often, but I try to go to GitHub and look at the PR.

Interviewer: When you accept a suggestion or solution from an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Yes, yes, I definitely verify it. Take the electronic HR documents I mentioned: when it wanted to go with Paperless-ngx, there were also some ideas around Mayan, but that had its problems too. So I explored Paperless-ngx myself.
Interviewer: Let's say you're using AI on a project over the long term. Do you think that as time goes on, the AI gives you more accurate output or makes the process smoother? Or does it stay as it is, or get worse?

Participant: As time goes on, new features keep being added to the project. That depends completely on my initial architecture decisions. Sometimes I started working without thinking about the fact that one feature would depend on another. The problem then is that the agent breaks my known architecture and known codebase to try to fit them to the other, dependent feature. Then my own code becomes unfamiliar to me. That causes some trouble. When I'm chasing bugs, I see: this used to be like this, so why was this added here? Because of that feature. That's the kind of trouble that happens.

Interviewer: Then what's the solution? How do you mitigate it?
Participant: To mitigate it, I go back to the discussion phase. There, I mention the dependent feature and ask the agent what it will do, how it will break it apart and how it will relate the two. By looking at how it relates them, I can often tell, for instance, that this was originally built for one client, or one category, or one year, and now the same kind of thing is coming in again. It says it will use a factory design pattern or something similar here. Then I know that, yes, it can handle it.

Interviewer: Looking back on your AI-assisted development over these years, has it been helpful for you, is it creating problems for you, or do you think it will create problems in the future?

Participant: For me or for everyone?

Interviewer: For you.

Participant: For me, it has been very helpful. I've found common ground where I can easily reach a shared conclusion that would otherwise have taken me a bit more time. And since I now have plenty of experience at the coding level, I don't have much trouble understanding things at the code level. But at the architecture decision level, it has been very helpful for me.

P19:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: Basically, I use multiple tools together. The one I've used most is the free version of Cursor. Along with that, I also use the free version of ChatGPT. I use the two in combination. Say I'm going to work on a specific module in a project. First, I use the web version to ask questions and learn general information about the module. Then I go to the tool. When there's a time constraint and I have to finish the work within a certain time, I go to the agentic tool. Also, when there's a somewhat bigger task where I can do some parts by hand and hand other parts to the agent, that's when I mainly use this tool.
Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: I gave part of that answer just now. Let me explain in more detail. Say I'm going to work on a specific existing project. I have a time limitation there. Since everyone in the industry knows it's possible to do this within that time if i have AI, there's time pressure on me; even if I wanted to, I couldn't do the whole thing myself. So the primary reason for me is time.
Second, I have to see whether having the agent do it would take less effort from me. And the effort I would have spent coding it myself to get the output, I can put somewhere else: it could be team discussion or research on other things. So, based on my analysis, I hand the agent the task and tell it how it should be done, and meanwhile I work on something else. There's an element of multitasking here; that's basically when I use agentic coding.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?
Participant: There isn't a single answer, because sometimes it has given me an output I didn't ask for that I really liked. So I discussed it with my team afterwards and kept it. But in some cases it has also misinterpreted my instruction, and then kept making the same mistake again and again. That happens too. So you have to work with a mix of both. When I like it, I keep it; when I don't, I have it reviewed again.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: That also depends on which tool I'm using. If I use a better-quality tool, it has various flag systems, meaning various command systems. Using those, I can keep it in planning

mode. It will generate the plan first, and after generating it, when a question comes up about how to do something, it asks me: "How should I do this? Which one do you want?"
There's one more thing here: it often misses important questions that it should have asked me, but didn't. Then I have to check on my own. Even with a better model or tool, it's hit and miss: it may ask, or it may not.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: There's really no way to know that in advance; at least I can't. The way I go about it is: first I analyze my requirements. I note down the requirements for that specific module in a notebook, or brainstorm a bit and make a flow. After making the flow, I have it generate a plan. Then it asks whatever questions it has, and if the plan needs updates, I give input and have it updated. That's when a standard plan comes together.
Then I have a teammate or my senior verify it. When they validate it and say "you can proceed," I tell the agent to do the work based on that plan. About 90% of the time it gets it done fairly accurately, but sometimes it causes trouble: sometimes it doesn't understand the instruction and goes off in another direction. So you have to keep an eye on those things while it's working. You mostly realize it after seeing the output; it's not very easy to tell beforehand.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yes, I've faced that many times. If I don't document it properly or keep personal notes, backtracking becomes very difficult. Not just for me; my teammates often find it difficult to backtrack too. Then you have to understand it again from the start: how the data is moving, which record is flowing where. That part specifically becomes time-consuming. So yes, that happens.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?
Participant: To be honest, I've done both. At first, when the boom happened, it was new to me, just like everyone else. Back then I would just try it and see what output it gave. I didn't know the standard way of working that has more or less taken shape now. So at first, I did that too. What I do now is: if there's something I don't know, if I don't know what a function does, I go through the existing project with it a bit to understand what's happening there. When it explains, it takes me less time to understand than working it out on my own. Once I understand, I explain

to it, the existing part is like this, and I need to build the feature in this place. Then it becomes easy for me to decide how I'll go about the work. So that's basically how I proceed now.
Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: It kind of depends on who is reviewing the code. If I'm reviewing it, it's based on my experience: it might seem to me that this looks right and works properly. But when someone with a bit more experience than me looks at it, they might see that there could be a security problem here, or a best-practice problem. So it's probably easier for people with more experience. I don't know; that's just how it seems to me.
Interviewer: When you accept a suggestion or solution from AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Yes, it has to be cross-checked, because it often happens that it misinterprets what I said or takes it in the wrong direction. Then I have to do a Google search and check an existing blog or the documentation to see whether it's roughly correct.
Something that would have been very hard for me to understand from the documentation alone, it gives me a vague idea of in a short time. I then match that against the documentation: this is what I got from the AI, and this is what the documentation says. That makes the documentation easier for me to read, and if there's a mistake, I spot it. I note it down and later ask it to correct it. When I tell it, "You made this mistake here," it often understands what the issue was. It fixes it and gives a new plan, which I check again, and then I move the work forward.

Interviewer: Let's say you're using an AI tool in your project, and the context inside the project keeps growing, meaning you keep using it on the same project for a long time. Does that affect the code output? Because of the long use, is it giving better output or worse output?

Participant: Yes, I've felt that. Theoretically, I'd heard that it has an effect later, that it shouldn't stay in that much context. Practically, while working myself, I've faced it: often, in fact most of the time, it doesn't answer properly; it analyzes things wrongly, or does the specific task in the project wrongly. Then things go wrong. So you have to keep an eye on the context and make sure it doesn't get too long. You just have to watch that.

Interviewer: If you look back at this phase of AI-assisted development, has it been helpful for you? Or do you see it as a threat? From which side do you see it?

Participant: Honestly, I see it both ways. When I'm working, it feels like it's helping a lot. But at the same time, it also crosses my mind that just doing this kind of work won't be enough; I need to improve myself more, because before long it will be able to do these jobs by itself. That's on

my mind; maybe it crosses everyone's mind at some point that I need to get better, because it's already taking over these sectors. Yes, I definitely keep that in mind.

P20:

Interviewer: What AI coding tools do you currently use, and do you use free or paid versions? Do you mainly use conversational tools, where you provide specific code or questions, or agentic tools that can access and modify a larger part of your codebase?

Participant: I use Cursor as an agentic coding tool. I don't just ask questions with Cursor; it's an agentic tool, and it has access to a large part of my code, everything, to modify it.

Interviewer: At what point in a task do you decide to hand over control to an AI agent instead of doing the work yourself? What usually influences that decision?

Participant: All the work is given to it right from the very start. First in planning mode, to see what it actually wants to do. Then, if it gives a proper plan, good. If not, I tweak the plan, change some things, modify it the way I want, or use the existing best practices to tell the tool to update it. Once it gives the plan, the next steps go ahead according to that plan.

Interviewer: Can you describe a situation where an AI agent did more than you originally asked it to do? What did it change, and how did you respond to those additional changes?

Participant: In this case, it has had both positive and negative effects. Suppose I ask it to write code for some features; sometimes it actually writes a lot more code. Sometimes that helps. For example, there are many scenarios where I forget to mention handling certain cases, and it handles them on its own, automatically. But in other cases, it writes random, unnecessary extra functions that actually increase code complexity. So after having the agent write the code, you have to review it and fix it properly.

Interviewer: When a requirement, business rule, or edge case is unclear, how does the AI usually respond? Does it ask for clarification, or does it make an assumption and continue?

Participant: First, it tries to work by assuming things on its own. Different models behave differently here. Some models, the high-thinking ones, actually ask me follow-up questions about how to handle these cases, and they even give options: A, B, C, D. So I choose from those. And when I see that it's hallucinating instead of asking, I tell it that in cases like this it shouldn't hallucinate and work without understanding, but should ask me questions instead.

Interviewer: When asking an AI agent to implement a task, how do you make sure it understands what you want and why?

Participant: First of all, as I said earlier, I actually give it to the agent in planning mode. While planning, it tells me itself whether its requirements match mine. Then I make the plan to start the

task, that is, I plan before coding, so that I avoid the kind of mistake where I wanted one thing and it returns something else as the result.

Interviewer: Can you remember a situation where you or another developer later had to change AI-generated code but didn't know the original reason behind the implementation? What happened?

Participant: Yes, yes, this situation has happened many times. There’s a solution for it, if I ask the agent, it tells me again why it implemented it that way.

Interviewer: When you have to work on something you don't know well, how do you use AI? Do you first ask it to explain the concept and then implement it, or do you generally let it handle the implementation for you? What determines your choice?

Participant: I first ask it to explain. After it explains, some more research happens on top of that, through communicating with it. Then the work starts.

Interviewer: Before accepting AI-generated changes into your project, how do you decide that the code is "good enough" to keep?

Participant: In that case, to decide whether the code is good enough, you actually have to review the code.

Interviewer: When you review it, do you base that on your experience, or on something else?

Participant: On experience, plus, when something seems illogical, I research it in other ways.

Interviewer: When you accept a suggestion or solution from an AI agent, do you usually verify that it is correct before using it? If so, how do you verify it?

Participant: Verifying is actually based entirely on research. When it makes a decision that something should be done a certain way, if there's a new topic, or if it wants to implement something with a new tool, then I have to research it and check whether what I'm doing is right.

Interviewer: Let's say you're using AI tools continuously on a project.

Participant: Yes.

Interviewer: So, from using it continuously, as time goes on, does the AI give better results, or is it the same as before?

Participant: No, no, definitely, the more context goes into it, the better, nicer and more precise the results it gives. The more idea it has about the project, the better.

Interviewer: Looking back at your experience with AI-assisted development, what is the biggest problem or risk you've personally experienced that you think developers may overlook when using AI?

Participant: In the future, refactoring could be the biggest trouble. It writes code its own way, so you can't understand the code's complexity: what it wrote and how. So at first glance, a lot of it makes you wonder why it wrote this and what it wrote. So maintainability suffers. If the code is written with design patterns, then it's fine. But if those aren't maintained properly, there's a possibility of a technical hazard in the future.